

Computer Arithmetic

Session 1 – Adders

Introduction

What is Computer Arithmetic ?

- Pentium Division Bug



The image displays two side-by-side screenshots of a Windows 95 desktop environment, illustrating the Pentium Division Bug. Both screenshots show the Windows Calculator application with the formula $(4195835 / 3145727) * 3145727 - 4195835$ entered. In the left screenshot, the result is 0, with a red circle around the display and the text "We get 0, as expected" and "This should equal 0". In the right screenshot, the result is -256, with a red circle around the display and the text "Incorrect result!". Both screenshots also show the CPU-Z application, which identifies the processor as an Intel Pentium 90, B5 stepping. The text "Try it out!" is overlaid on the left screenshot, and "Pentium 90, B5 stepping" is circled in red at the bottom of the left screenshot. The text "Same formula..." is overlaid between the two screenshots, and "Pentium 90, B1 Stepping" is circled in red at the bottom of the right screenshot.

Calculator

We get 0, as expected

This should equal 0

Try it out!

Pentium 90, B5 stepping

Same formula...

Incorrect result!

Pentium 90, B1 Stepping

Introduction



What is Computer Arithmetic ?

- Pentium Division Bug

cost Intel a pre-tax charge of \$475 million (\$891 million in 2024) to replace faulty processors. The flaw, located in the Pentium's floating-point unit (FPU), caused inaccurate calculations in specific division operations

Introduction

What is Computer Arithmetic ?

- Scope of Computer Arithmetic

Hardware

Design of efficient digital circuits for primitive and other arithmetic operations such as $+$, $-$, $\times$, $\div$, $\sqrt{}$, $\log$, $\sin$, and $\cos$

Issues: Algorithms
Error analysis
Speed/cost trade-offs
Hardware implementation
Testing, verification

General-Purpose

Flexible data paths
Fast primitive operations like
 $+$, $-$, $\times$, $\div$, $\sqrt{}$
Benchmarking

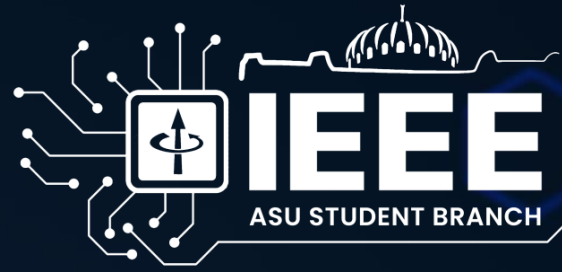
Special-Purpose

Tailored to application areas such as
Digital filtering
Image processing
Radar tracking

Software

Numerical methods for solving systems of linear equations, partial differential equations and so on

Issues: Algorithms
Error analysis
Computational complexity
Programming
Testing, verification

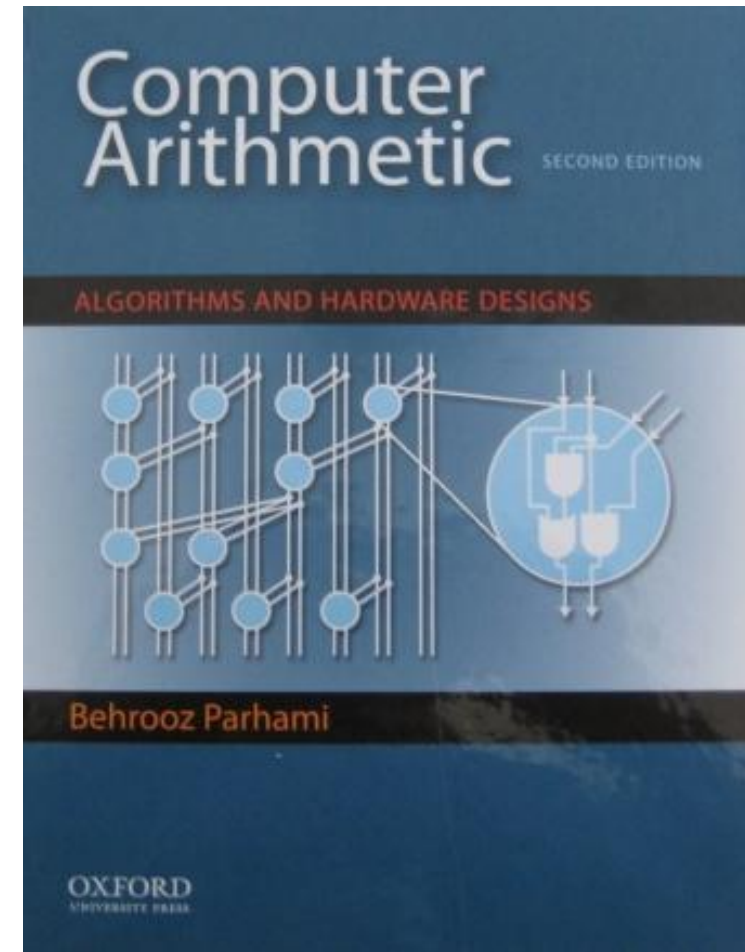


Course Overview

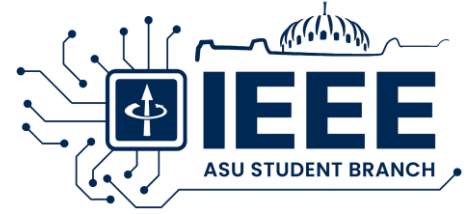
Overview

Main Textbook :

- B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd edition, Oxford University Press, New York, 2010.
(ISBN 978-0-19-532848-6, 641+xxv pp., 316 figures, 718 problems)



Overview



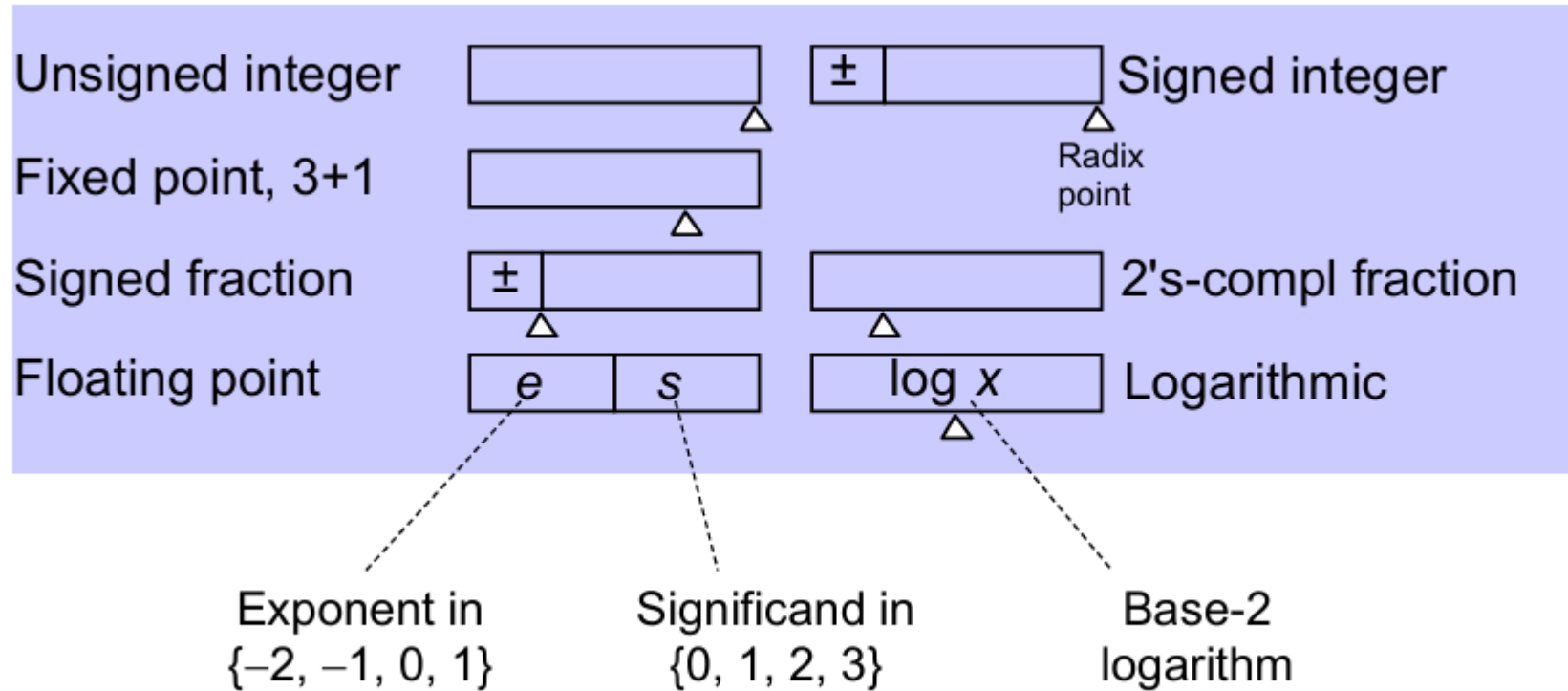
Covered Topics :

- Introduction to Number Representation
- Addition / Subtraction
- Multiplication
- Division
- Real Arithmetic
- Function Evaluation

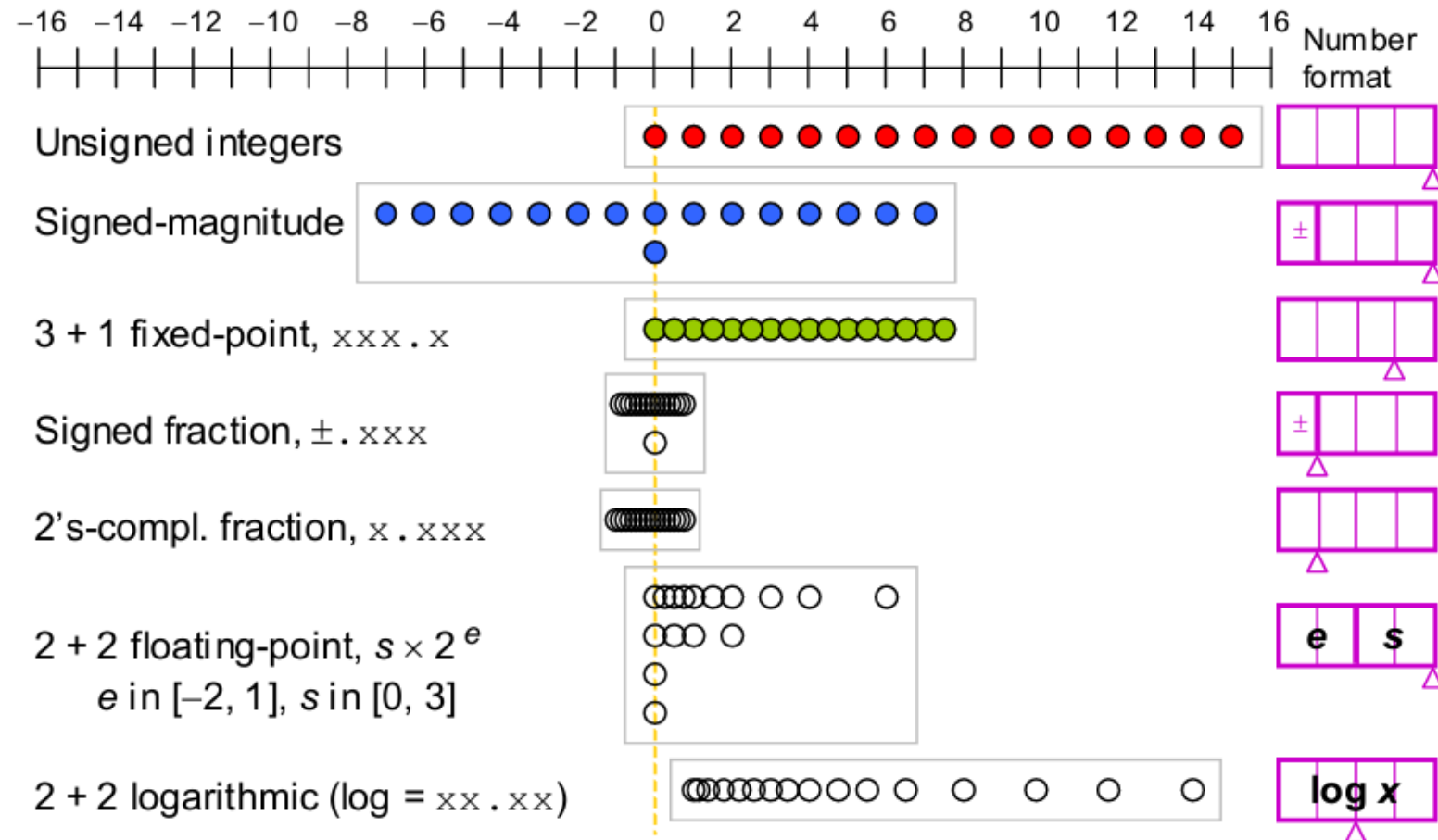


Number Representation

Encoding Number

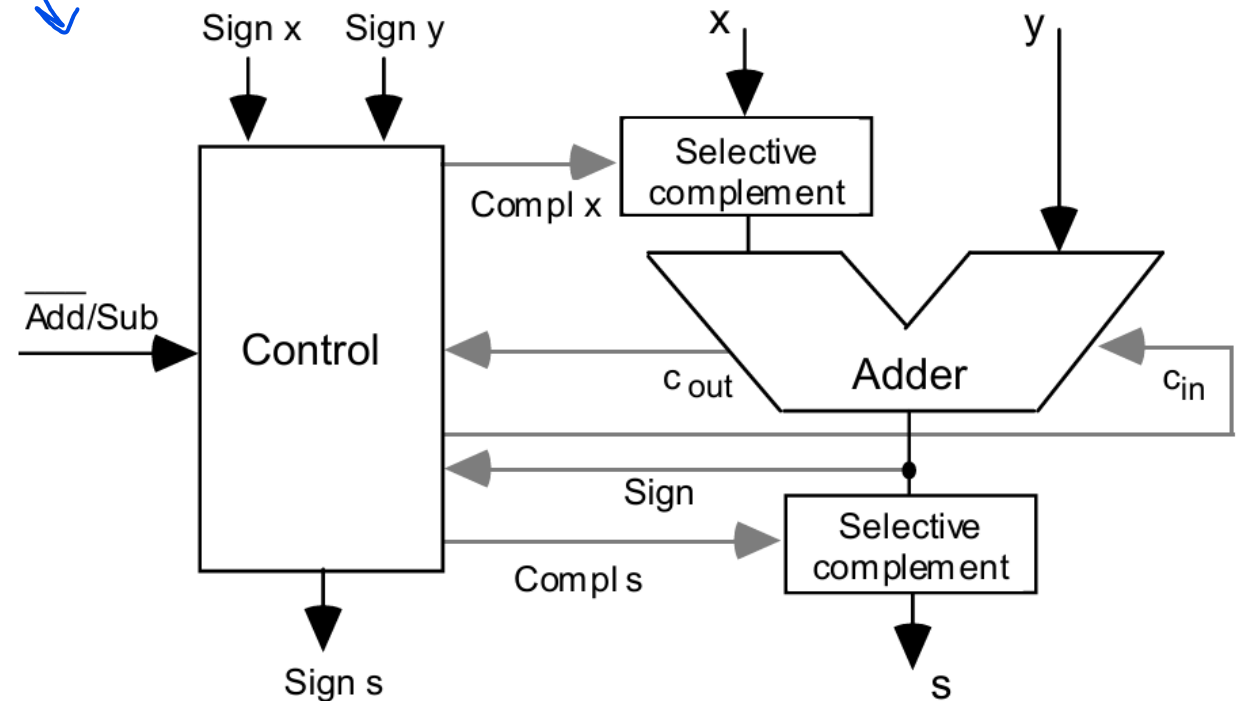
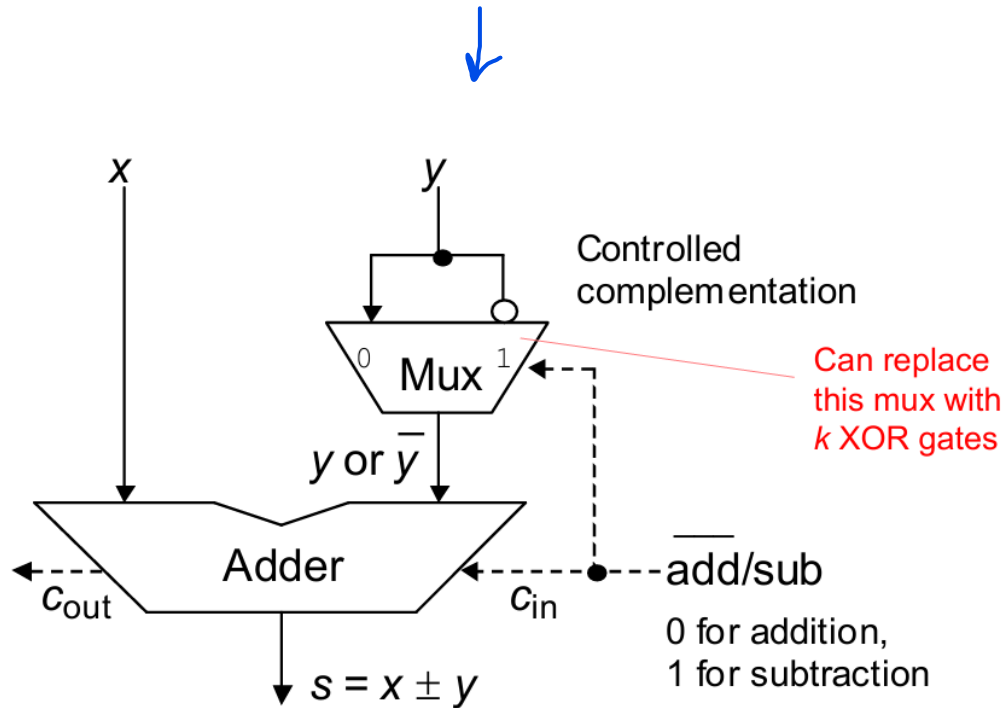


Encoding Number

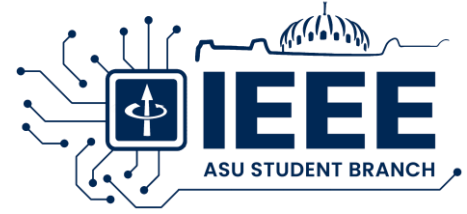


Sign Representation

2's-Complement Vs Signed-Magnitude Adder



Dot Notation



A Useful Visualization Tool

- ● ● ● ● ● ● ● Unsigned positive-radix number
- ● ● ● ● ● ● ● 2's-complement number
- ● ○ ● ○ ● ○ ● Negative-radix number

Dot Notation

A Useful Visualization Tool

$$\begin{array}{r} + \quad \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \\ \hline \bullet \bullet \bullet \bullet \bullet \end{array}$$

(a) Addition

(b) Multiplication

$$\begin{array}{r} \times \quad \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \\ \hline \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \\ \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \\ \hline \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \bullet \end{array}$$

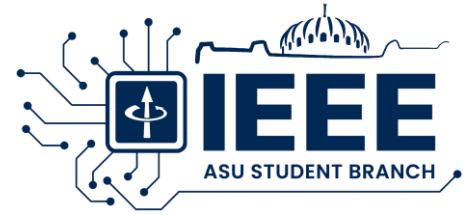
Number Representation



Adders

Introduction

Addition



Binary

$$\begin{array}{r} \\ \\ \\ \hline (12) \end{array}$$

Binary addition of (21) and (-9) resulting in (12). The numbers are represented in 8-bit two's complement form. The carry bits (1s) are shown above the digits.

Decimal

$$\begin{array}{r} \\ \\ \\ \hline \end{array}$$

Decimal addition of 4653 and 2167 resulting in 6820. The carry digits (1s) are shown above the digits.

Addition

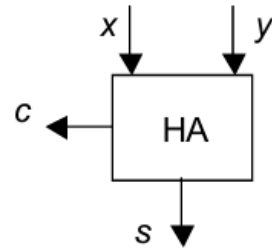
Dot Notation

$$\begin{array}{cccccc} & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ & \bullet & \bullet & \bullet & \bullet & \bullet \\ + & & & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \\ 2^5 & & & & & \end{array}$$

$$\begin{array}{cccccc} & -2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ & \bigcirc & \bullet & \bullet & \bullet & \bullet \\ + & & & \bigcirc & \bullet & \bullet \\ \hline \bigcirc & \bullet & \bullet & \bullet & \bullet & \bullet \\ -2^5 & & & & & \end{array}$$

Single bit Addition

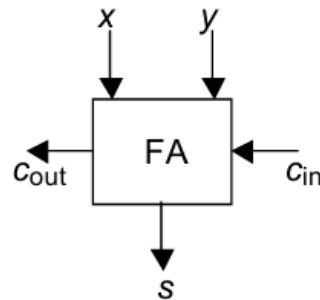
Inputs		Outputs	
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



$$S = X \oplus Y$$
$$C = X.Y$$

Half-adder (HA): Truth table and block diagram

Inputs			Outputs	
x	y	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

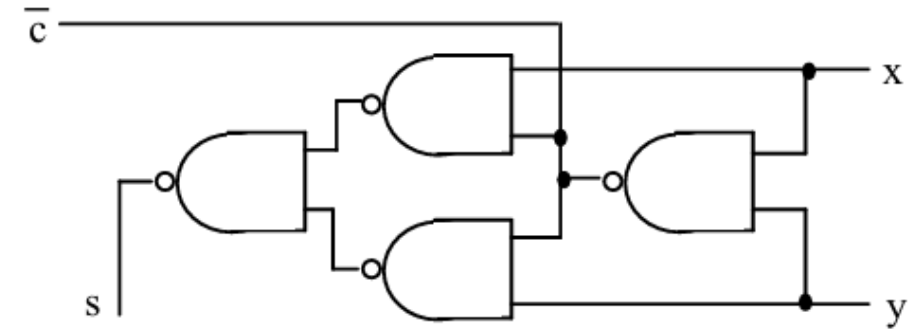
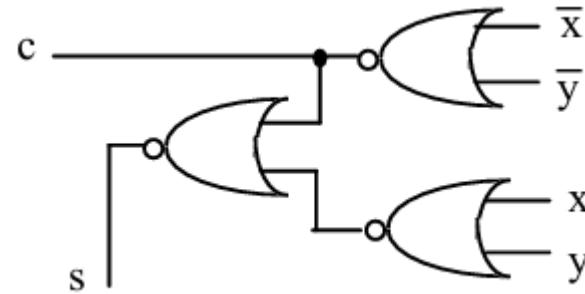
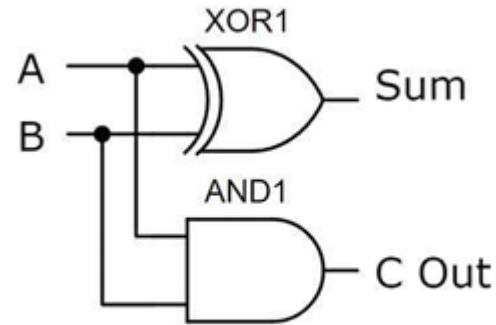


$$S = X \oplus Y \oplus c_{in} \quad (\text{Odd Parity Function})$$
$$C_{out} = (X.Y)V(X.c_{in})V(Y.c_{in}) \quad (\text{Majority Function})$$

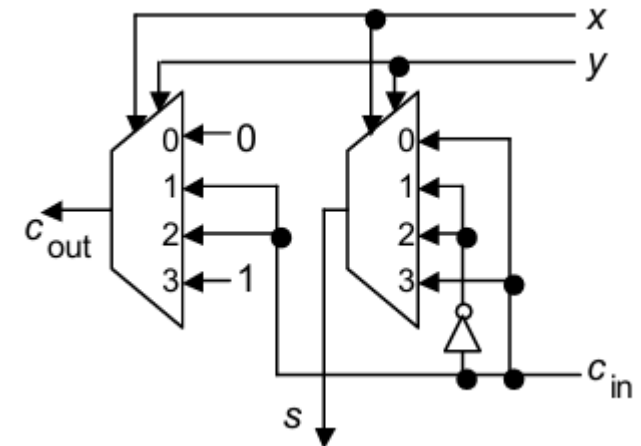
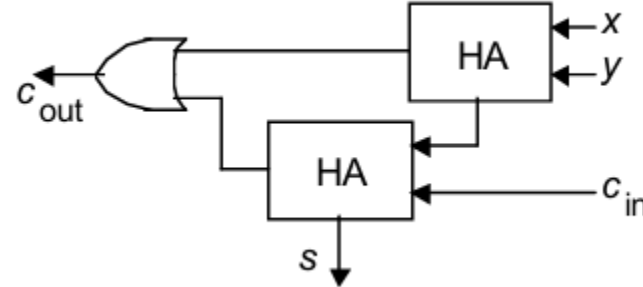
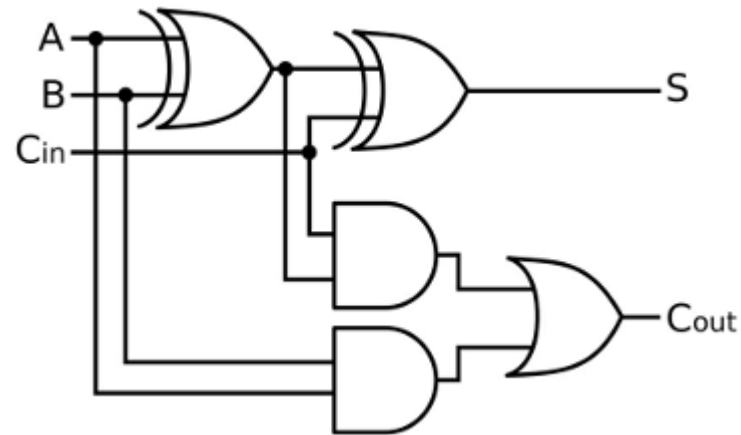
Full-adder (FA): Truth table and block diagram

Single bit Addition

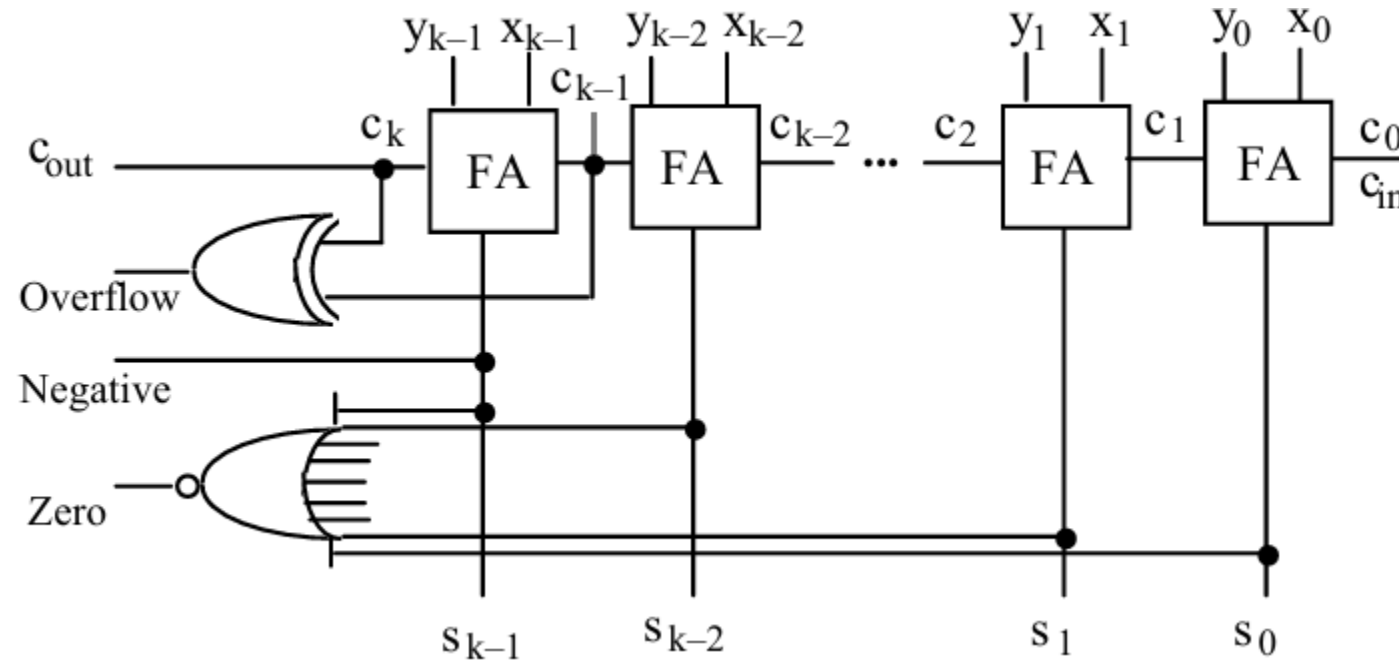
Half Adder



Full Adder



Addition Flags

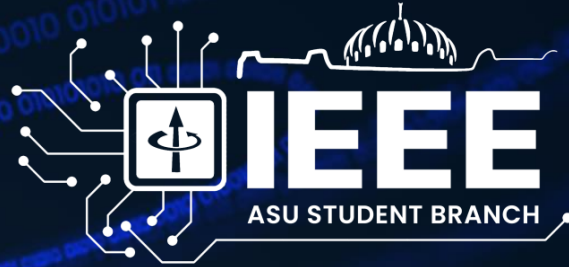


- A number (S) not in the range $[S_{min}:S_{max}]$ is represented by $\text{Sum mod } 2^N$.

Note: N is Number of bits

$$\text{overflow}_{2's\text{-compl}} = x_{k-1} y_{k-1} s_{k-1}' \vee x_{k-1}' y_{k-1}' s_{k-1}$$

$$\text{overflow}_{2's\text{-compl}} = c_k \oplus c_{k-1} = c_k c_{k-1}' \vee c_k' c_{k-1}$$



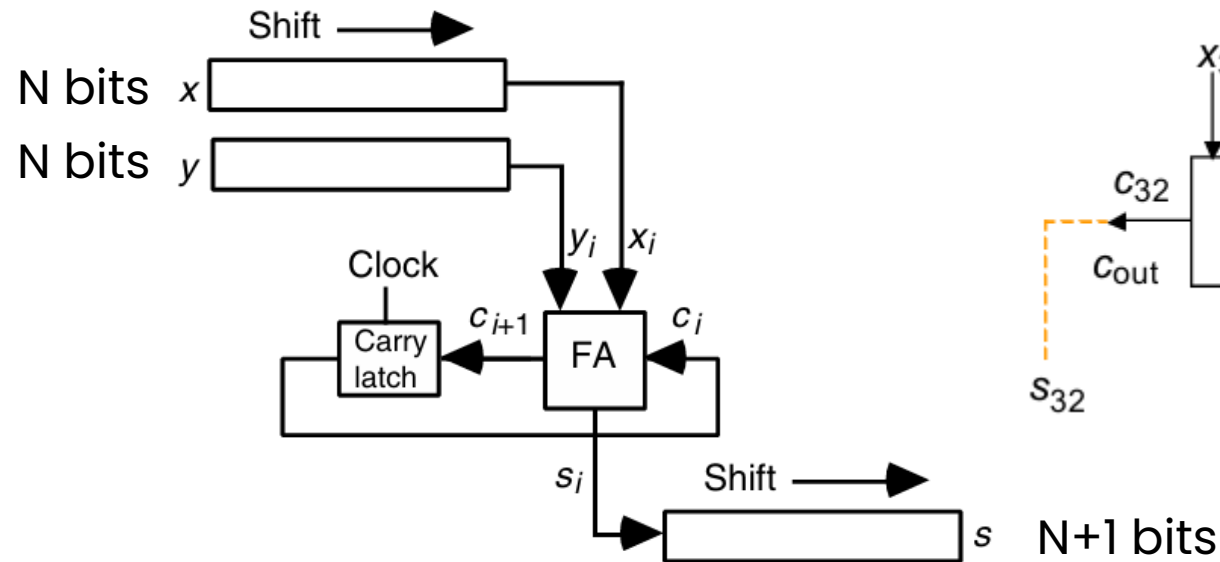
Adders

Basic Addition

Multi-bit Addition

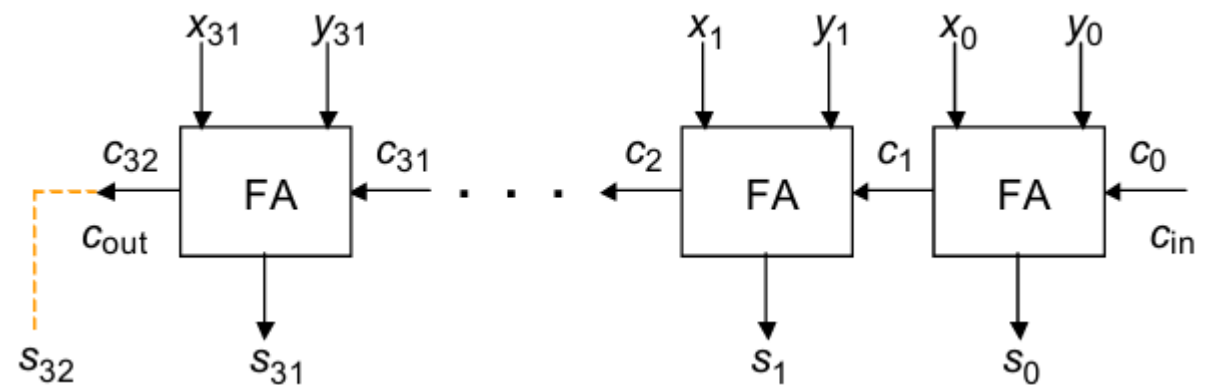
Using Full Adders in Building Addition Blocks

Bit-Serial Adder



- Low Hardware Cost
- Requires N clock cycles (high latency)
- Total computation time comparable to RCA (for N-bit addition)
- But RCA is better, because it completes the Addition in a single cycle.

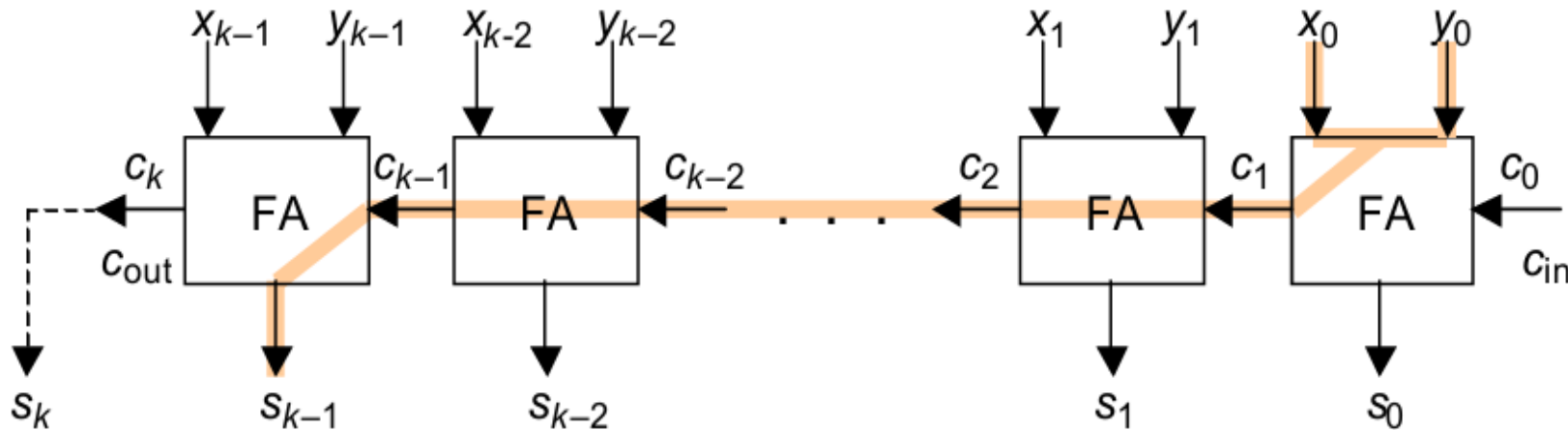
Ripple Carry Adder (RCA)



Critical Path Through a RCA

What is the Critical Path ?

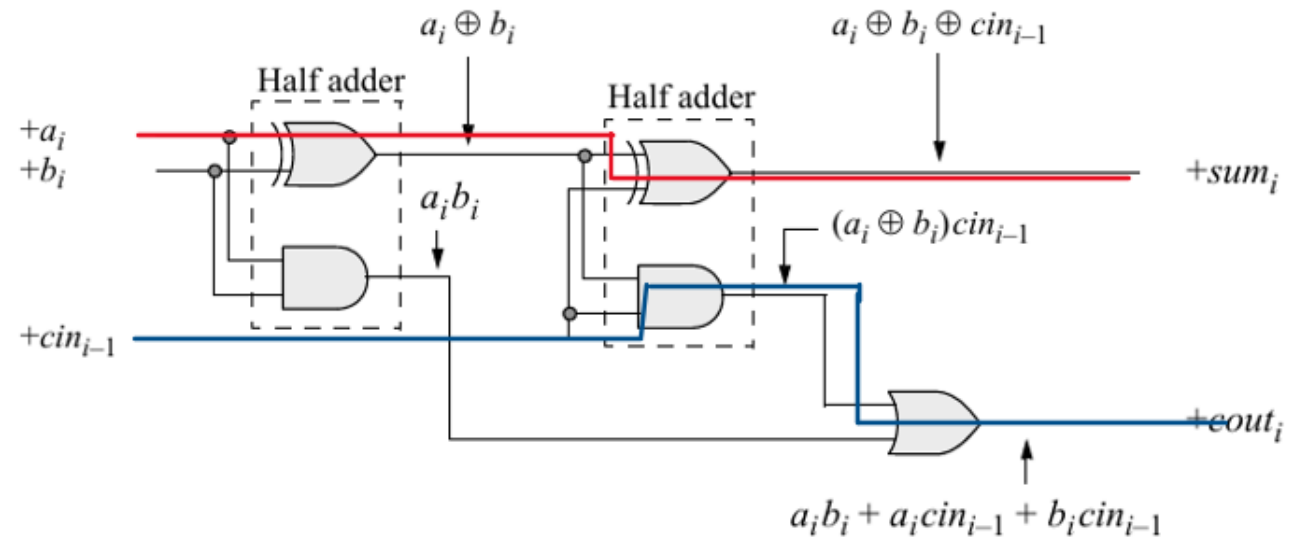
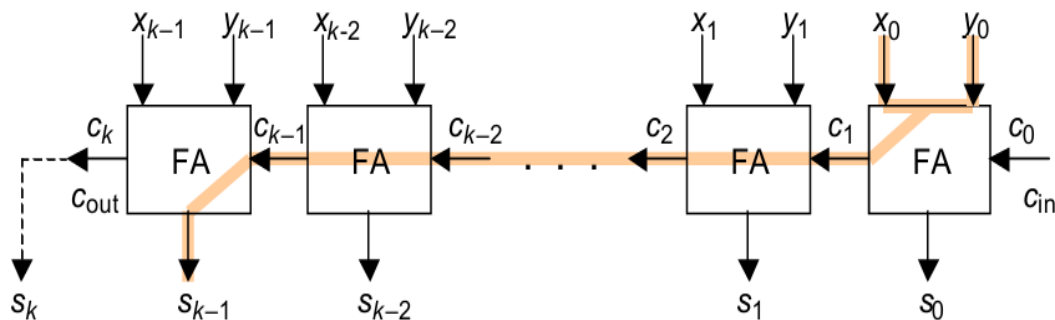
- Critical path in RCA goes through FAs, The original path, It starts at input X_0 and Y_0 goes through all the intermediate FAs and end at the last sumbit.
- There are many kinds of adders, The main Problem in adders designs is The Carry Problem, It is the most problem and limit the speed.



Critical Path Through a RCA

What is the Critical Path ?

Why not critical path the carry path does not start the worst case got a path from C_{in} to C_{out} ?



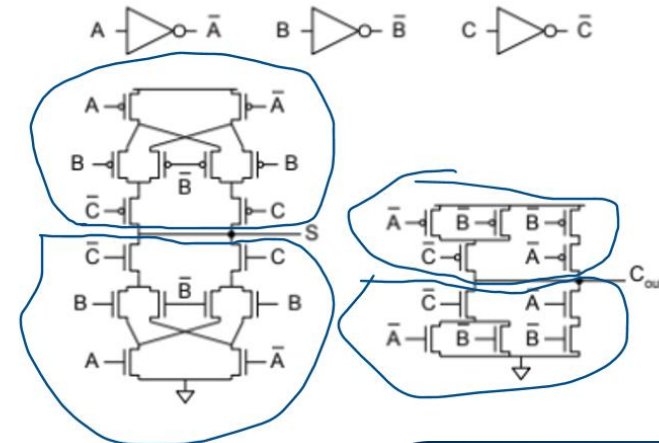
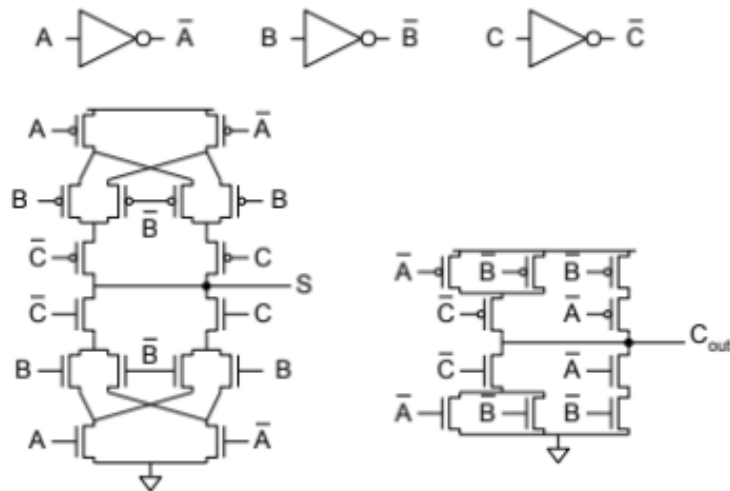
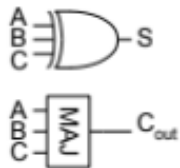
Basic Addition

Critical Path Through a RCA

Full Adder Design 1 Custom

- Self-dual function: inverting inputs inverts outputs, this is important for Inverting characteristics of CMOS Circuits.
- The XOR gate is one of the most complex and slowest gates, and this creates a performance bottleneck in adders.
- 32 Transistors (6 for the inverters, 10 for the majority gate, and 16 for the 3-input XOR)
- Problem is Big Area 32 Transistors, If I have 64 FAs!!

$$S = A \oplus B \oplus C$$
$$C_{out} = \overline{A}B + \overline{C}(\overline{A} + \overline{B})$$
$$= \text{MAJ}(A, B, C)$$

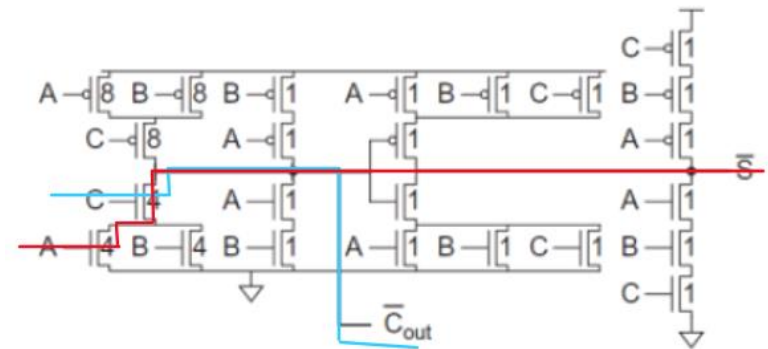
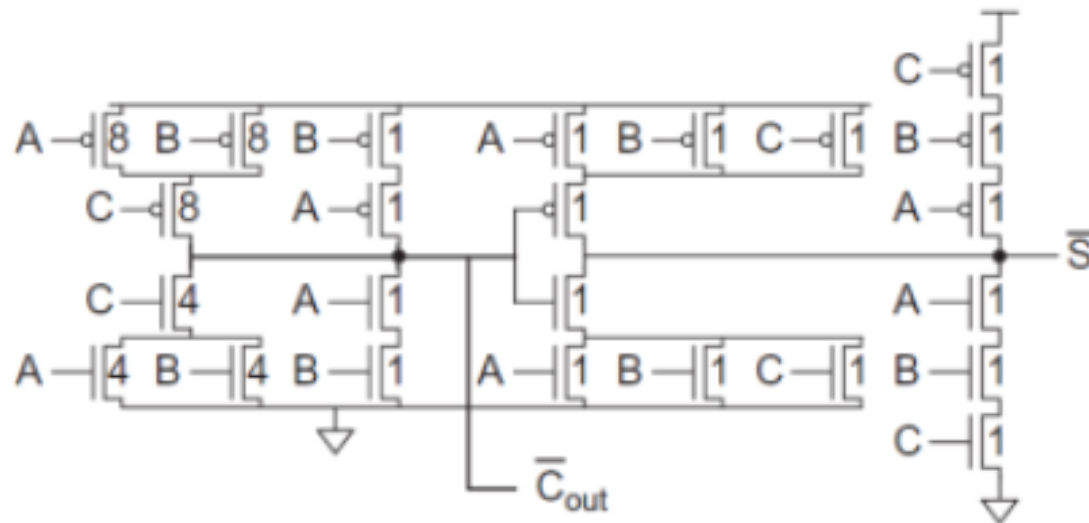


Basic Addition

Critical Path Through a RCA

Full Adder Design 2 Custom

- Combine S in terms of C_{out}
- $S = ABC + (A + B + C)(\sim C_{out})$
- Self-duality: PMOS network is identical to NMOS network rather than being the conduction complement (mirror adder)
- Critical path is usually C_{in} to C_{out} in ripple adder (extra delay computing S is unimportant), so it is most important to speed path from C_{in} to C_{out} .



Basic Addition

Critical Path Through a RCA

Delay and Cost Expressions

- The expression $T_{\text{ripple-add}} \approx k \cdot T_{\text{FA}}$ shows that the delay of a ripple-carry adder increases linearly with the number of bits k . This is because each full adder must wait for the carry from the previous stage, creating a carry propagation chain. Thus, the overall latency grows proportionally with k , which is expressed as $O(k)$. So does bit serial adder is $O(k)$ although the constant of proportionality is larger here because of the latching and clocking overheads.

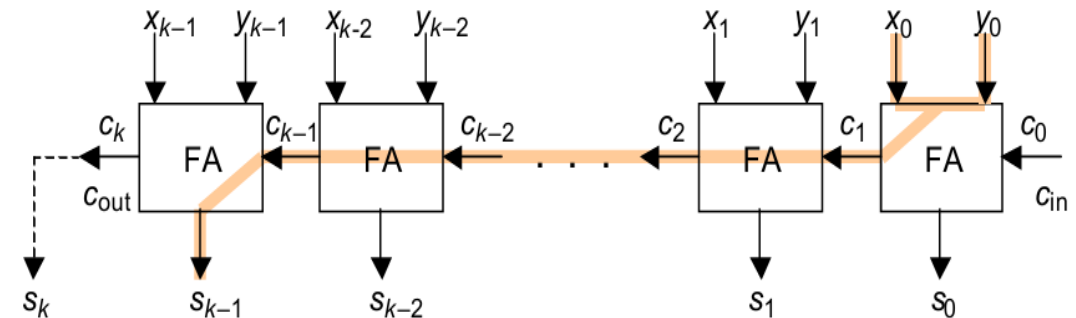
$$T_{\text{ripple-add}} = T_{\text{FA}}(x, y \rightarrow c_{\text{out}}) + (k - 2) \times T_{\text{FA}}(c_{\text{in}} \rightarrow c_{\text{out}}) + T_{\text{FA}}(c_{\text{in}} \rightarrow s)$$

$$\text{Area}_{\text{bit-serial}} = n \cdot A_{\text{FA}}$$

$$T_{\text{bit-serial}} = n \cdot T_{\text{FA}}(c_{\text{in}} \rightarrow s)$$

$$\text{Area}_{\text{bit-serial}} = A_{\text{FA}} + (3n+1) \cdot A_{\text{FF}}$$

$$\text{Thruput}_{\text{pipeline}} = 1/T_{\text{FA}}(c_{\text{in}} \rightarrow s)$$



Basic Addition



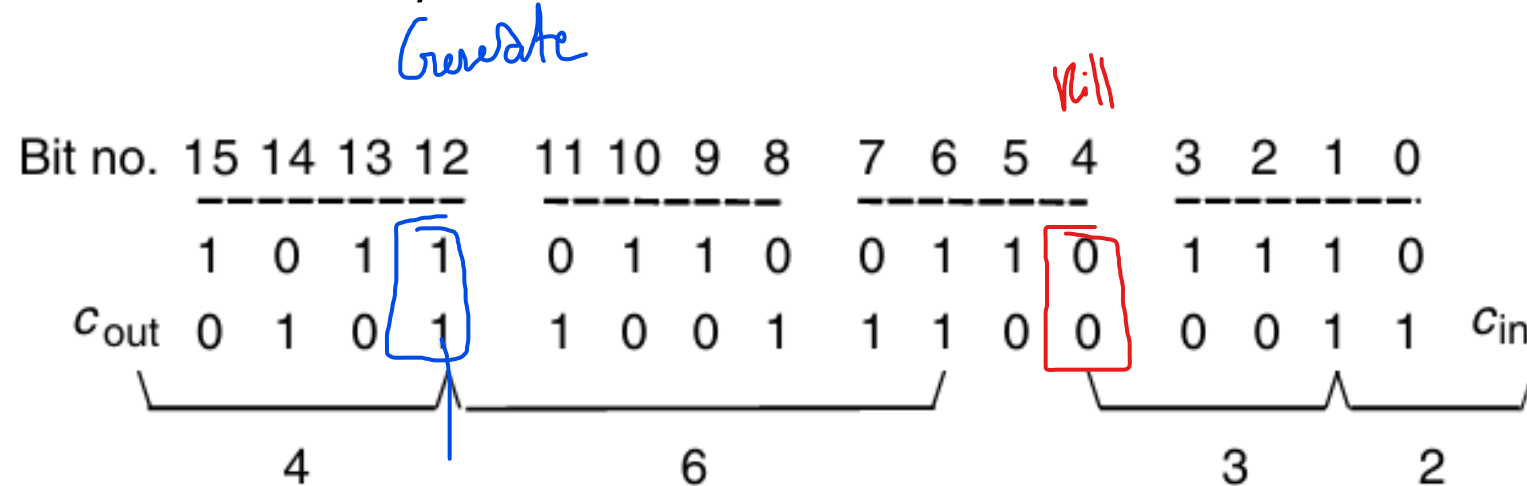
Adders

Intro to Fast Adders

Generate, Propagate, Kill

Analysis of Carry Propagation

- In adders, the main delay comes from carry propagation. To speed this up, we classify how each bit handles the carry using Generate, Propagate, and Kill.
- Each bit handles carry in one of three ways:
 - **Generate:** creates carry ($A \cdot B$)
 - **Propagate:** passes carry ($A \oplus B$)
 - **Kill:** removes carry ($A \cdot \bar{B}$)



Generate, Propagate, Kill

- Generate: $C_{out} = 1$ independent of C
 - $G = A.B$
 - Propagate: $C_{out} = C$
 - $P = A \oplus B$
 - Kill: $C_{out} = 0$ independent of C
 - $K = \bar{A}. \bar{B} = \overline{A \vee B}$
 - $S = A \oplus B \oplus C = P \oplus C$
 - $C_{out} = G \vee (P.C)$
 - $C_{out} = G \vee (\bar{K}.C)$
- P is XOR function and it is too slow, so I will use Kill instead of it to maximum speed between C to C_{out} .

A	B	C	G	P	K	C_{out}	S
0	0	0	0	0	1	0	0
		1				0	1
0	1	0	0	1	0	0	1
		1				1	0
1	0	0	0	1	0	0	1
		1				1	0
1	1	0	1	0	0	1	0
		1				1	1

Generate, Propagate, Kill

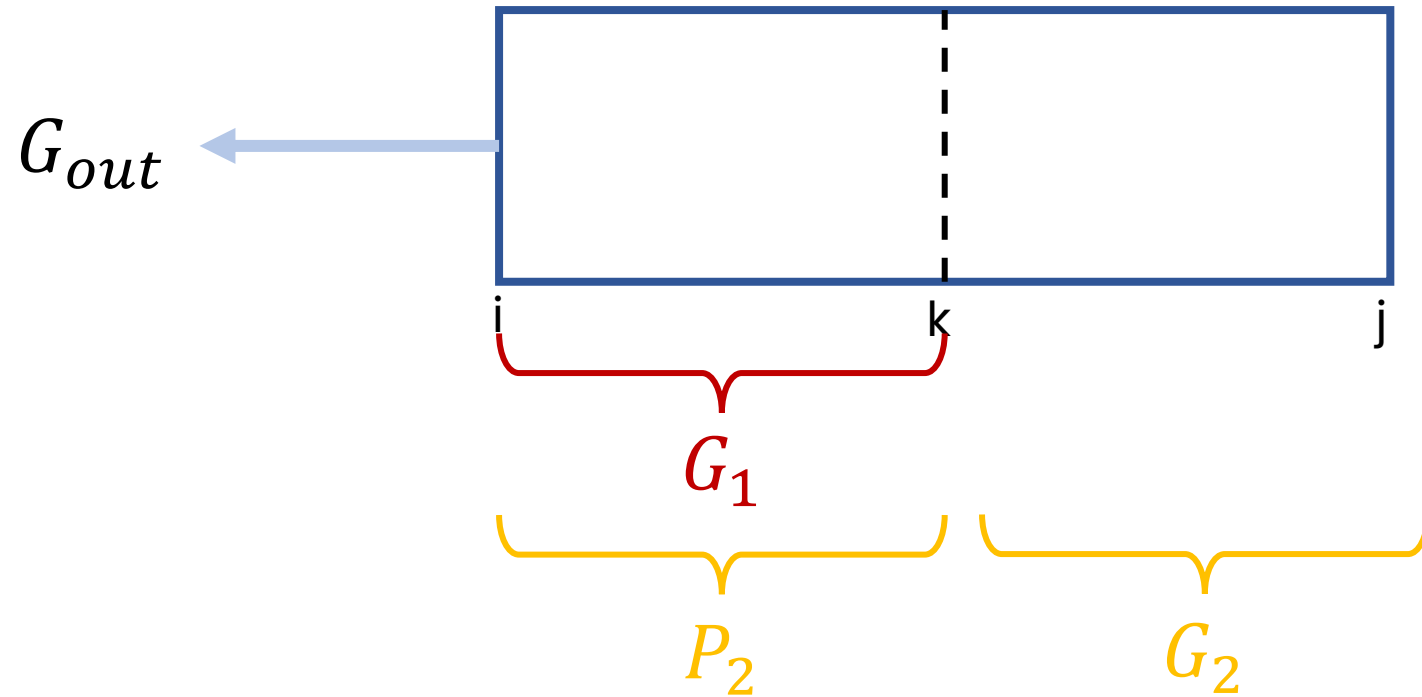


Group PG Signals

- Generalize G and P signals to describe a group of bits i:j.
 - A group of bits generates a carry if its carry-out is true independent of the carry-in.
 - A group of bits propagates a carry if its carry-out is true when there is a carry-in.
- Bit-wise generate and propagate signals
 - $G_{i:i} = G_i = A_i \cdot B_i$
 - $P_{i:i} = P_i = A_i \oplus B_i$
- Generate and Propagate for group of bits spanning i:j ($i \geq k > j$)
 - $G_{i:j} = G_{i:k} \vee (P_{i:k} \cdot G_{k-1:j})$
 - $P_{i:j} = P_{i:k} \cdot P_{k-1:j}$

Generate, Propagate, Kill

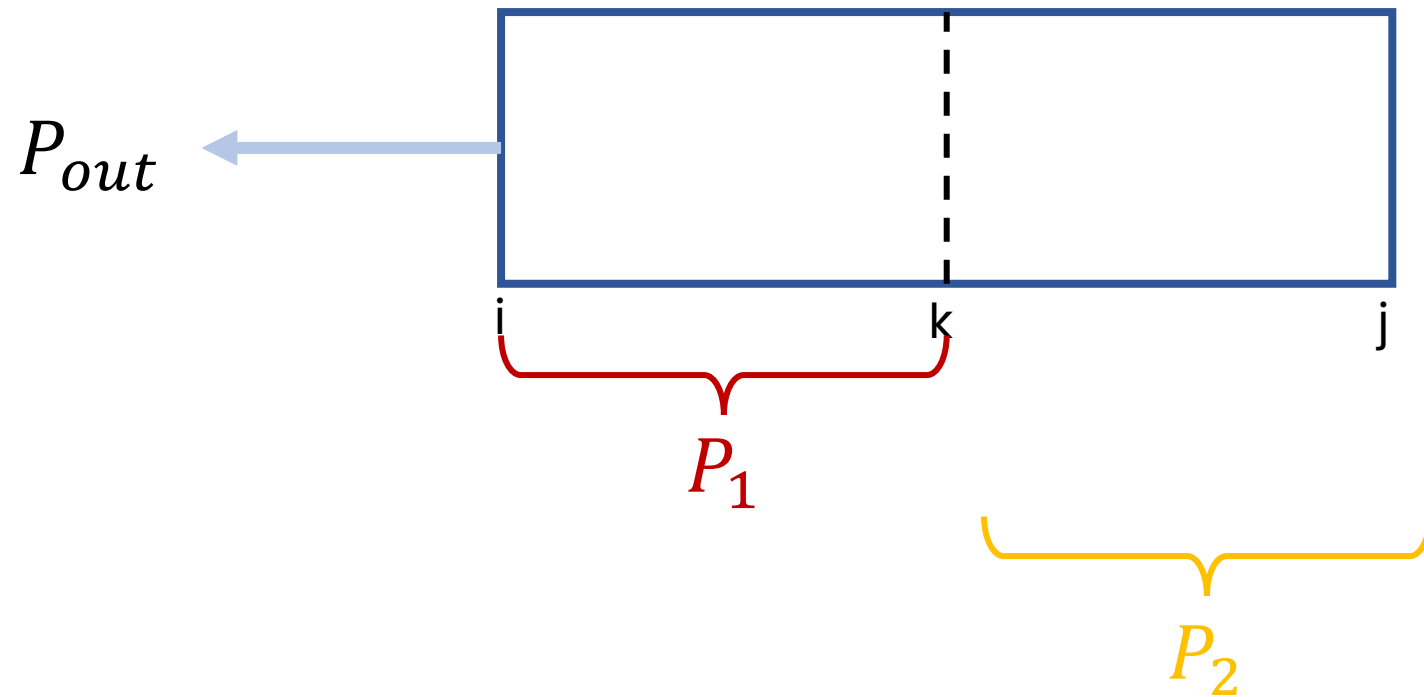
Group PG Signals



$$G_{out} = G_1 \vee G_2 P_2$$

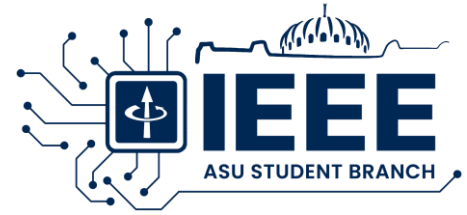
Generate, Propagate, Kill

Group PG Signals



$$P_{out} = P_1 \cdot P_2$$

Generate, Propagate, Kill



Group PG Signals (Examples)

- Note that: $C_i = G_{i:0}$
- Bits 1:0 ($i=1, k=1, j=0$)
 - $G_{1:0} = G_{1:1} \vee P_{1:1} G_{0:0} = G_1 \vee P_1 C_0 = G_1 \vee P_1 C_0 = C_1$

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j}$$

$$P_{i:j} = P_{i:k} P_{k-1:j}$$

- Bits 2:1 ($i=2, k=2, j=1$)

- Bits 2:0 ($i=2, k=2, j=0$)

-

- Note: $i=k$, Group P not required, AND-OR with pervious Group G (carry-in)

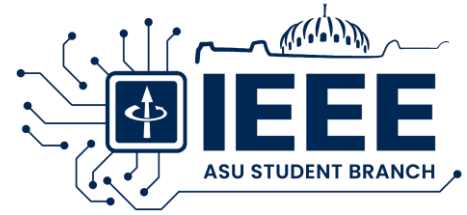
- Bits 2:0 ($i=2, k=1, j=0$)

-

-

- Note: $i > k$: Both group G and group P required

Generate, Propagate, Kill



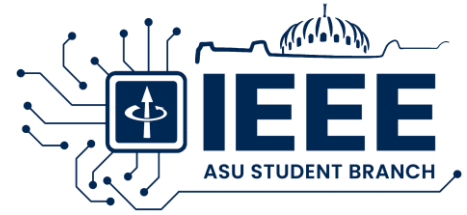
Group PG Signals (Examples)

- Note that: $C_i = G_{i:0}$
- Bits 1:0 ($i=1, k=1, j=0$)
 - $G_{1:0} = G_{1:1} \vee P_{1:1} G_{0:0} = G_1 \vee P_1 C_0 = G_1 \vee P_1 C_{in} = C_1$
- Bits 2:1 ($i=2, k=2, j=1$)
 - $G_{2:1} = G_{2:2} \vee P_{2:2} G_{1:1} = G_2 \vee P_2 G_1$
- Bits 2:0 ($i=2, k=2, j=0$)
 - $G_{2:0} = G_{2:2} \vee P_{2:2} G_{1:0} = G_2 \vee P_2 C_1 = C_2$
 - Note: $i=k$, Group P not required, AND-OR with pervious Group G (carry-in)
- Bits 2:0 ($i=2, k=1, j=0$)
 - $G_{2:0} = G_{2:1} \vee P_{2:1} G_{0:0} = (G_{2:2} \vee P_{2:2} G_{1:1}) \vee (P_{2:2} P_{1:1}) G_0 = (G_2 \vee P_2 G_1) \vee (P_2 P_1 G_0) = G_2 \vee P_2 (G_1 \vee P_1 G_0)$
 - $G_{2:0} = G_2 \vee P_2 C_1 = C_2$
 - Note: $i > k$: Both group G and group P required

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j}$$

$$P_{i:j} = P_{i:k} P_{k-1:j}$$

Generate, Propagate, Kill



Group PG Summary in 3 Steps

1. Base Case (bit 0 is the carry-in)

$$G_{\{i:i\}} = G_i = A_i \cdot B_i \quad G_{\{0:0\}} = G_0 = C_{\{in\}}$$

$$P_{\{i:i\}} = P_i = A_i \oplus B_i \quad P_{\{0:0\}} = P_0 = 0$$

$$C_i = G_{\{i:0\}}$$

2. Generate and Propagate for groups spanning i:j ($i \geq k > j$)

$$G_{\{i:j\}} = G_{\{i:k\}} + P_{\{i:k\}} \cdot G_{\{k-1:j\}}$$

$$P_{\{i:j\}} = P_{\{i:k\}} \cdot P_{\{k-1:j\}}$$

Note: K may replace p as OR is Fast in generate equation

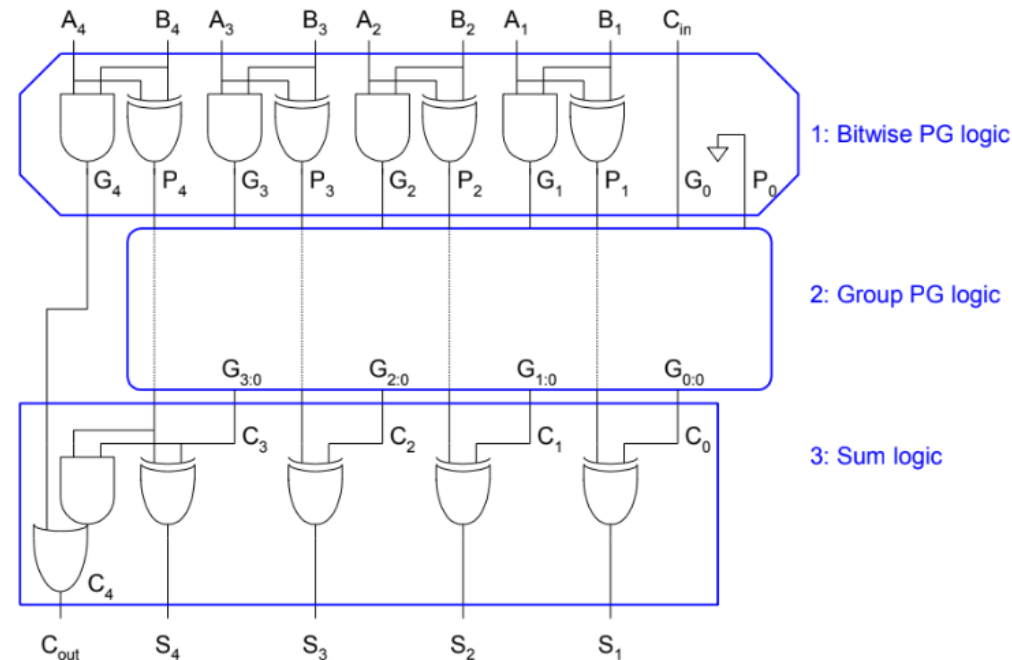
3. Sum: (*P is still required here, but off the critical path*)

$$S_i = A_i \oplus B_i \oplus C_{\{i-1\}} = P_i \oplus C_{\{i-1\}} = P_i \oplus G_{\{i-1:0\}}$$

Carry Network

Group PG Logic

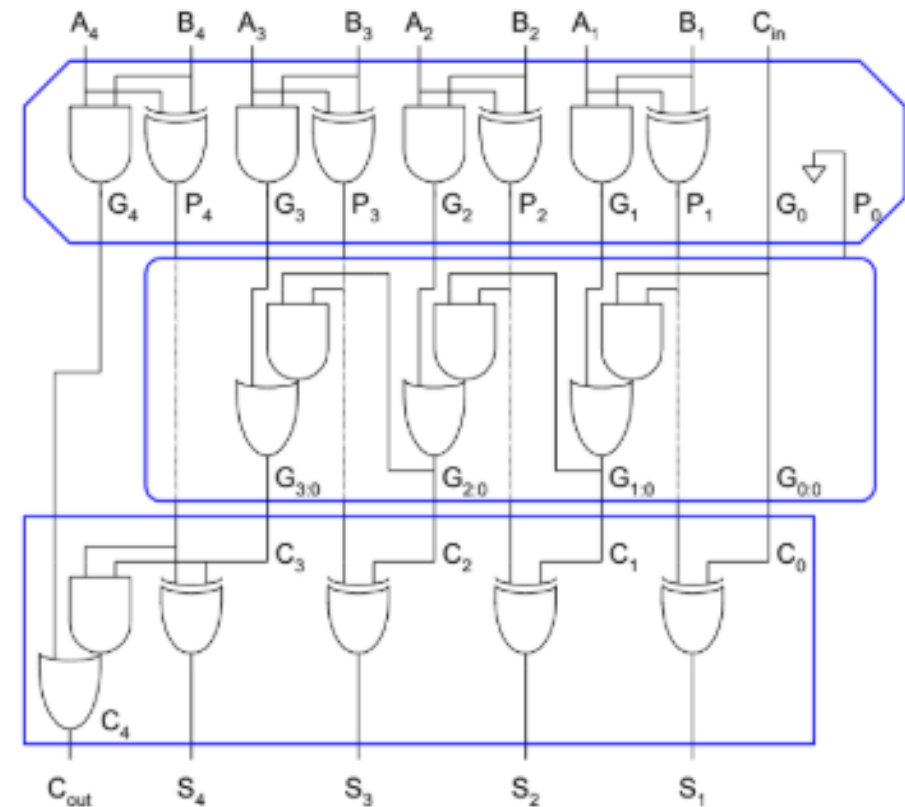
- *The first and third steps are routine.*
- *Most of the attention is devoted to alternatives for the group PG logic with different trade-offs between speed, area, and complexity.*



Carry Network

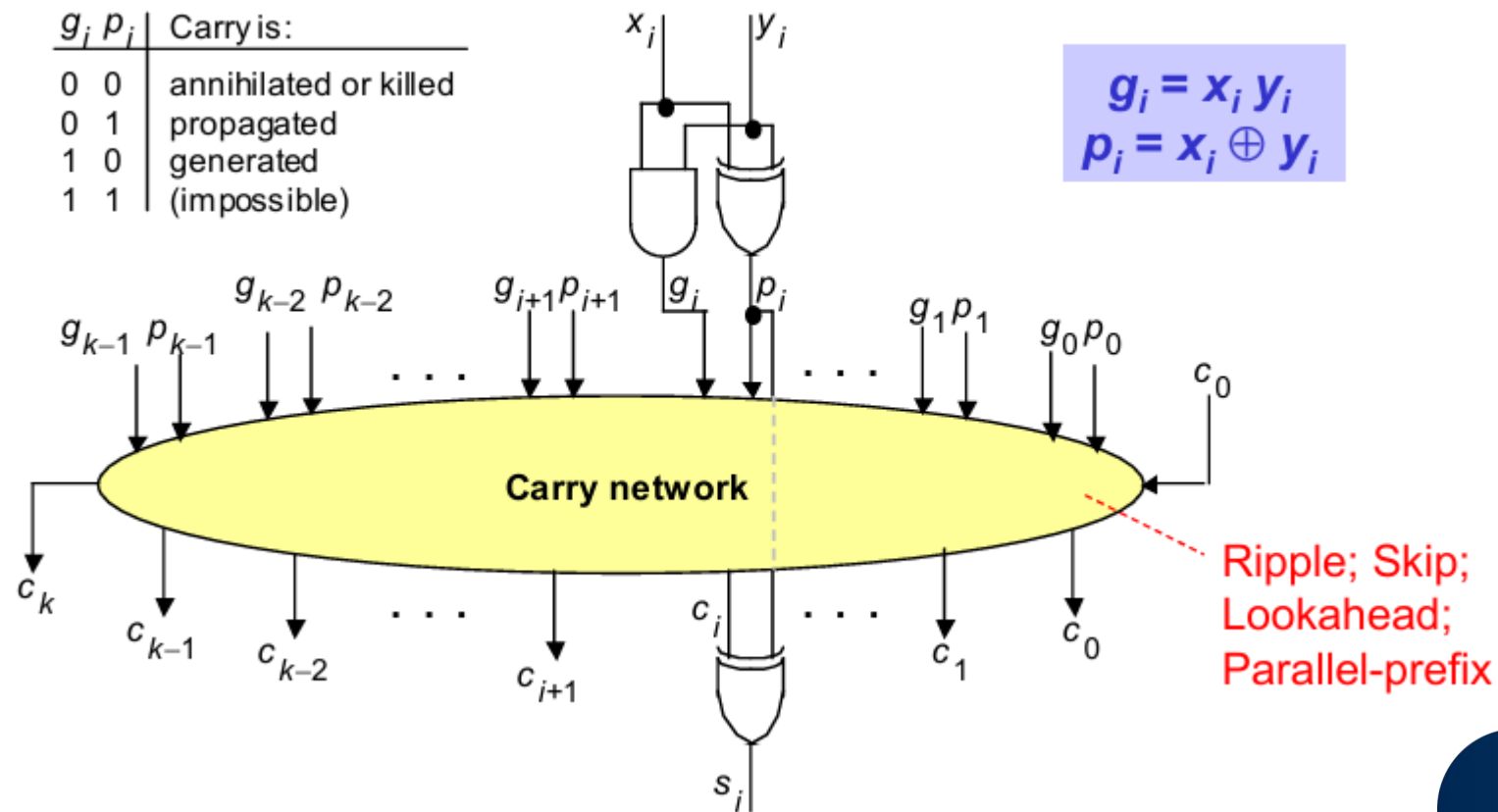
PG Carry Ripple

- $S_i = A_i \oplus B_i \oplus C_{\{i-1\}} = P_i \oplus C_{\{i-1\}}$
- For the carry from i to 0 to get C_i, set i = k:
- $G_{\{i:0\}} = G_{\{i:i\}} \vee P_{\{i:i\}}G_{\{i-1:0\}} = G_i \vee P_iC_{\{i-1\}} = C_i$
- *AND-OR current bit G with pervious Group G (carry-in)*
- *Group P not required*



Carry Network

Carry Network is the Essence of a Fast Adders

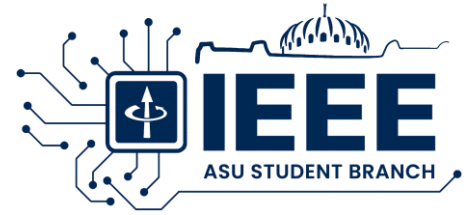




Adders

Carry-Lookahead Adders

From RCA to CLA



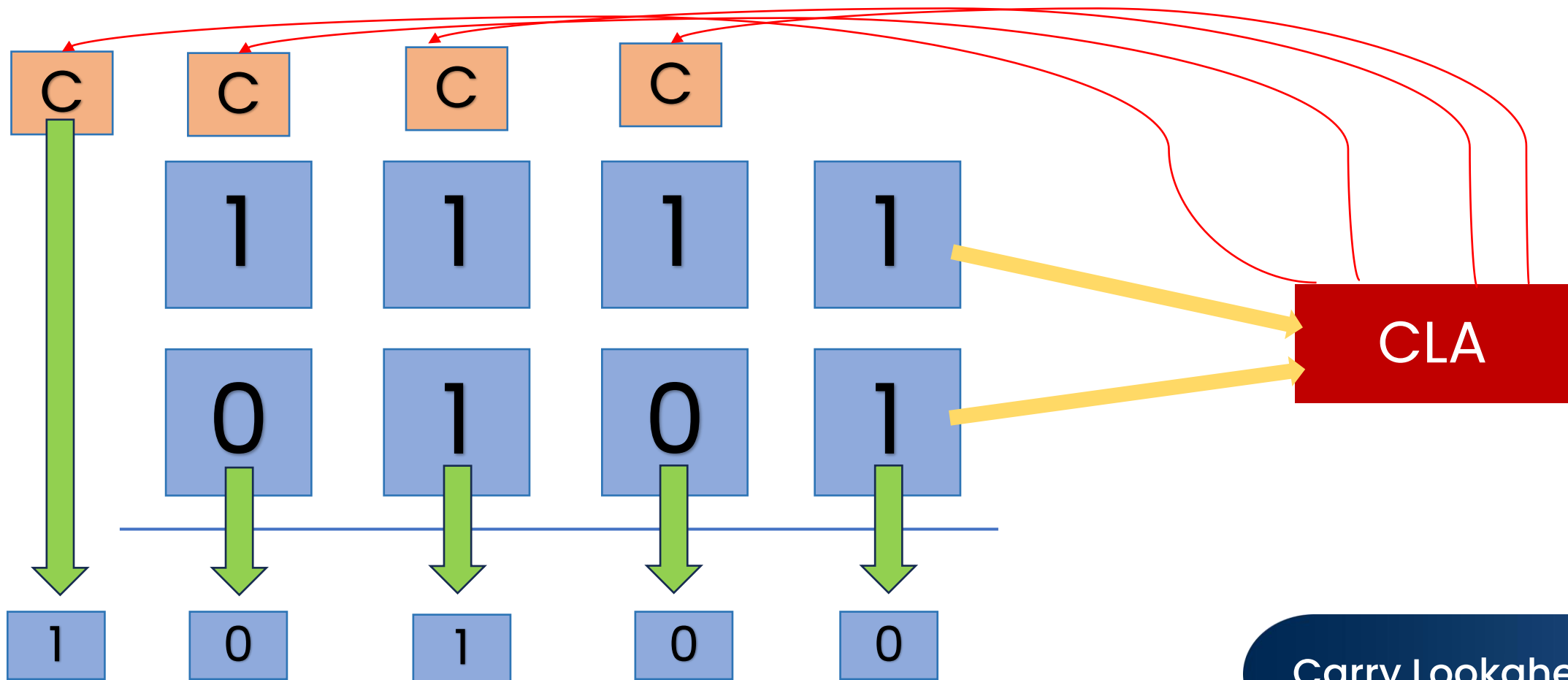
Speeding up Addition with carry lookahead

- Adders' designs considered before have worst-case delay that grows at least linearly with the word width k . An interesting question, therefore, is whether one can add two k -bit binary numbers in $O(\log k)$ worst-case time. carry-lookahead adders represent a commonly used scheme for logarithmic time addition.
- CLA achieves this speedup by exploiting parallelism: generate and propagate signals are computed simultaneously across multiple bit positions, allowing carries to be determined in parallel rather than rippling sequentially.

Carry Lookahead Adders

Concept of CLA

Lookahead: Parallel Generation of Carries



Carry Lookahead Adders

Unrolling The Carry Recurrence

The carry recurrence

$$c_{i+1} = g_i \vee p_i c_i$$

states that a carry will enter stage if it is generated in stage (i) or if it enters stage (i) and is propagated by that stage. One can easily unroll this recurrence, eventually obtaining each carry C_i as a logical function of the operand bits and C_{in} . Here are three steps of the unrolling process for C_i :

$$\begin{aligned} c_i &= g_{i-1} \vee c_{i-1} p_{i-1} \\ &= g_{i-1} \vee (g_{i-2} \vee c_{i-2} p_{i-2}) p_{i-1} = g_{i-1} \vee g_{i-2} p_{i-1} \vee c_{i-2} p_{i-1} p_{i-2} \\ &= g_{i-1} \vee g_{i-2} p_{i-1} \vee g_{i-3} p_{i-2} p_{i-1} \vee c_{i-3} p_{i-3} p_{i-2} p_{i-1} \\ &= g_{i-1} \vee g_{i-2} p_{i-1} \vee g_{i-3} p_{i-2} p_{i-1} \vee g_{i-4} p_{i-3} p_{i-2} p_{i-1} \vee c_{i-4} p_{i-4} p_{i-3} p_{i-2} p_{i-1} \end{aligned}$$

Unrolling The Carry Recurrence

Carry Interpretation (Unrolled Form)

- The expansion continues until the last term contains

$$c_0 = c_{in}.$$

- The unrolled carry equation has a simple meaning:

- Carry enters bit position i if:

- A carry is generated at position $i - 1 \rightarrow g_{i-1}$
- A carry generated at $i - 2$ is propagated by $i - 1 \rightarrow g_{i-2}p_{i-1}$
- A carry generated at $i - 3$ is propagated by $i - 2$ and $i - 1$

$$\rightarrow g_{i-3}p_{i-2}p_{i-1}$$

- ... and so on, back to c_{in}

Fast Carry Computation in a 4-bit Adder

- After full unrolling, all carries $c_1, c_2, \dots, c_k$ can be computed **directly** from:
 - **Generate signals:** g_i
 - **Propagate signals:** p_i
 - **Input carry:** c_{in}
- Uses TWO-Level AND-OR Logic
- Maximum gate fan-in: $K+1$
- For $K = 4$, the logic expressions are:

CLA Circuit



- For $K = 4$, the logic expressions are:

$$c_4 = g_3 \vee g_2 p_3 \vee g_1 p_2 p_3 \vee g_0 p_1 p_2 p_3 \vee c_0 p_0 p_1 p_2 p_3$$

$$c_3 = g_2 \vee g_1 p_2 \vee g_0 p_1 p_2 \vee c_0 p_0 p_1 p_2$$

$$c_2 = g_1 \vee g_0 p_1 \vee c_0 p_0 p_1$$

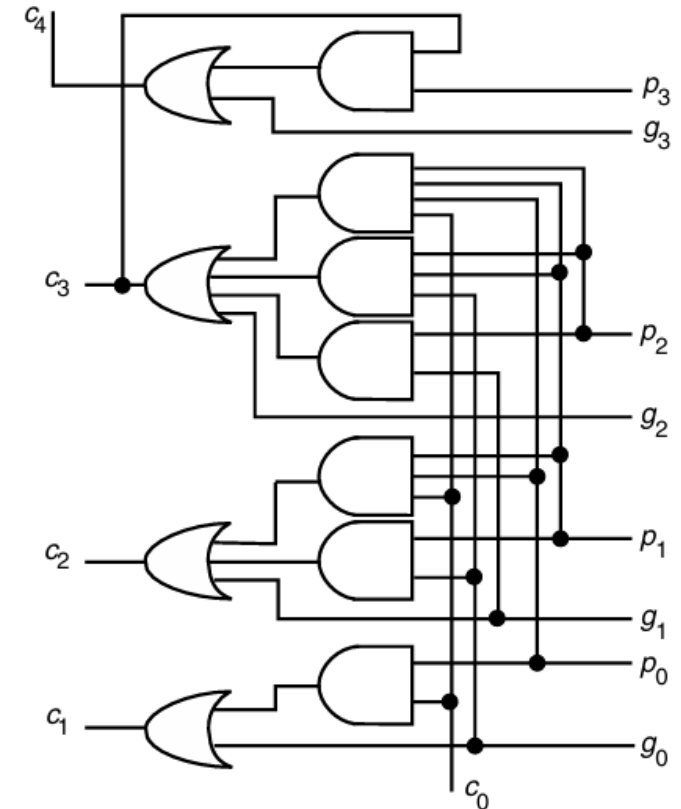
$$c_1 = g_0 \vee c_0 p_0$$

CLA Circuit

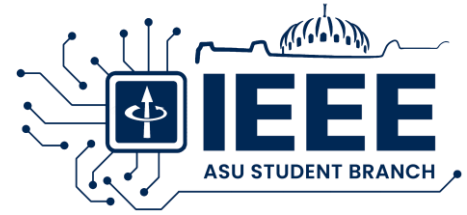
- A carry network based on the above equations can be implemented with less than two levels of two-input ANDs, producing the generate signals, and such as 8-input ORs, producing the lookahead; thus, it is possible to build a 4-bit binary adder. Such an adder is said to have **full carry lookahead**.

- $c_4 = g_3 \vee g_2 p_3 \vee g_1 p_2 p_3 \vee g_0 p_1 p_2 p_3 \vee c_0 p_0 p_1 p_2 p_3$
- $c_3 = g_2 \vee g_1 p_2 \vee g_0 p_1 p_2 \vee c_0 p_0 p_1 p_2$
- $c_2 = g_1 \vee g_0 p_1 \vee c_0 p_0 p_1$
- $c_1 = g_0 \vee c_0 p_0$
- Note: Since C_4 Doesn't affect the computation of the sum bits, it can be derived based on simpler equation:*

$$C_4 = g_3 \vee c_3 \cdot p_3$$



Multi-Level Lookahead



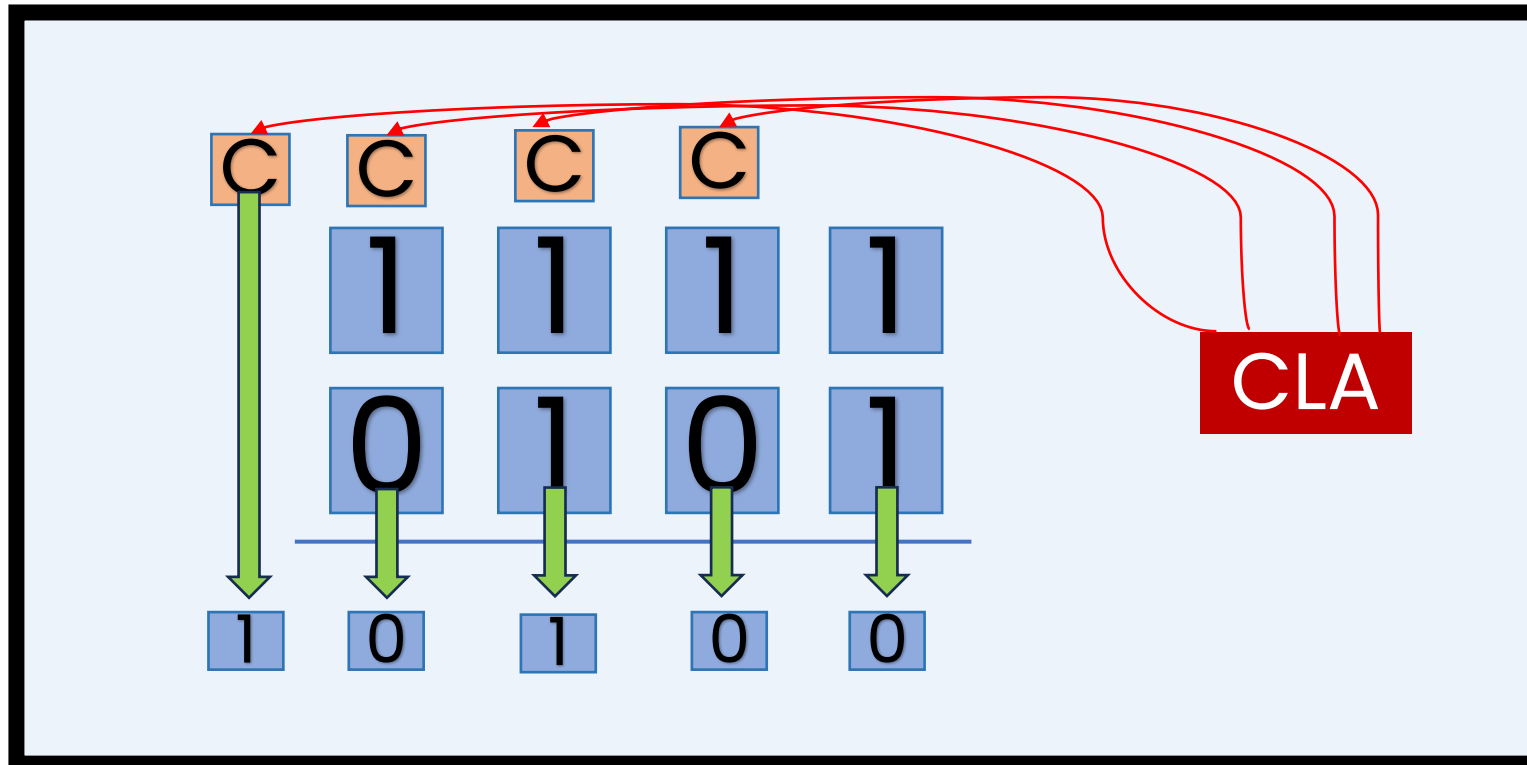
Carry Lookahead Complexity

- **Full carry lookahead** becomes impractical for wide words.
 - Example: c_{31} has **32 product terms**, the largest with **32 literals**.
 - Realizing the AND/OR functions requires **tree networks**, increasing **latency and cost**.
- **Two approaches to manage complexity:**
 1. High-radix addition (radix 2^h)
 - May increase latency for generating auxiliary signals and sum digits
 - Simplifies the carry network
 2. Multilevel lookahead
 - Technique used in **practical implementations**
 - Reduces network depth while keeping performance

Carry Lookahead Adders

Multi-Level Lookahead

Concept of Multi-Bit CLA

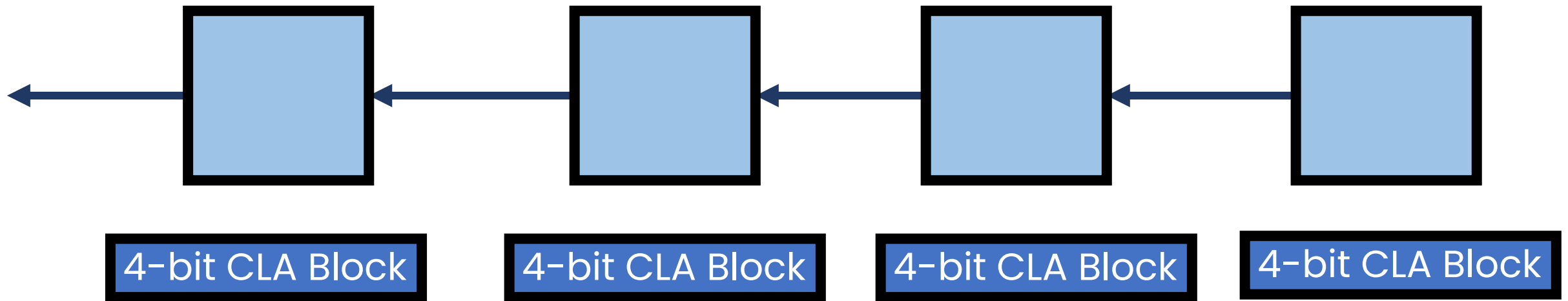


4-bit CLA Block

Carry Lookahead Adders

Multi-Level Lookahead

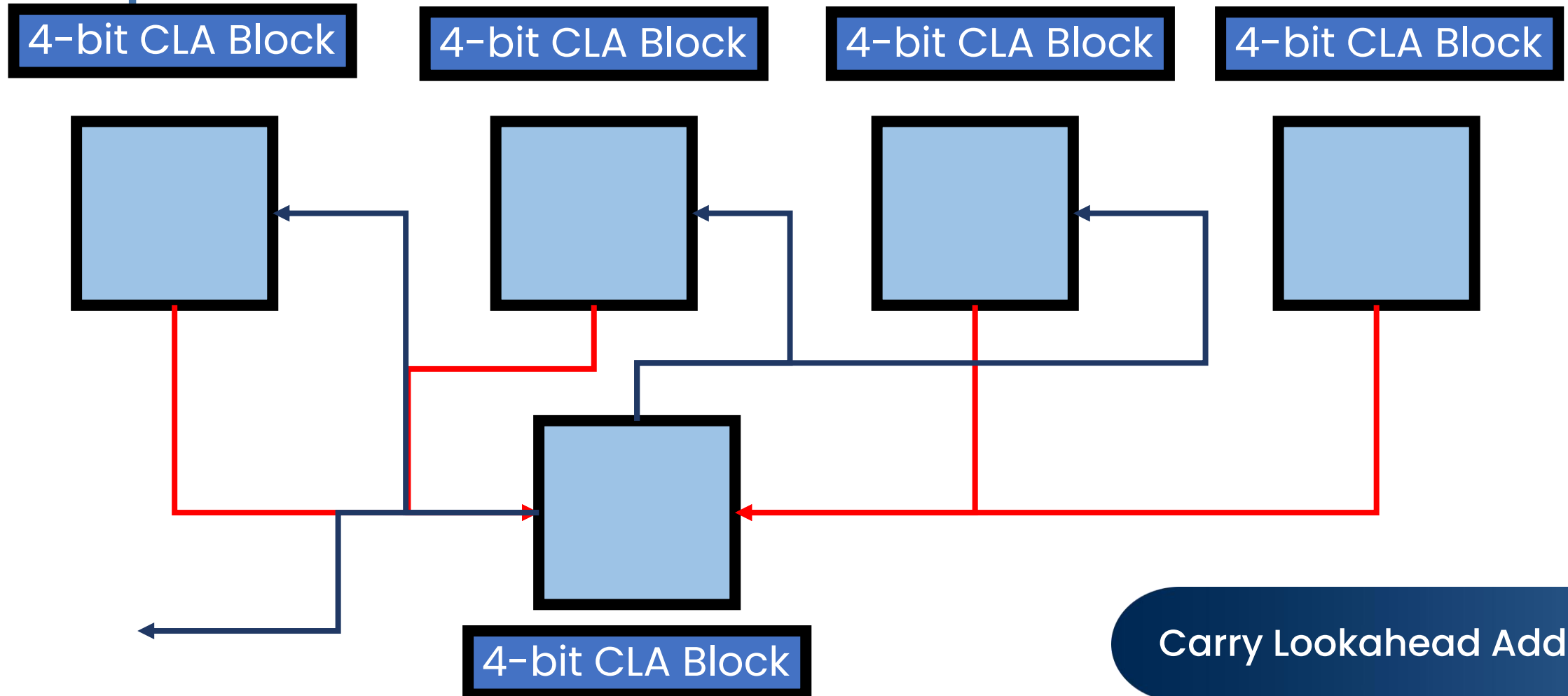
Concept of Multi-Bit CLA



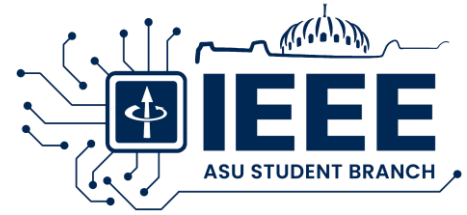
Carry Lookahead Adders

Multi-Level Lookahead

Concept of Multi-Level CLA



Multi-Level Lookahead



Block Generation & Propagation

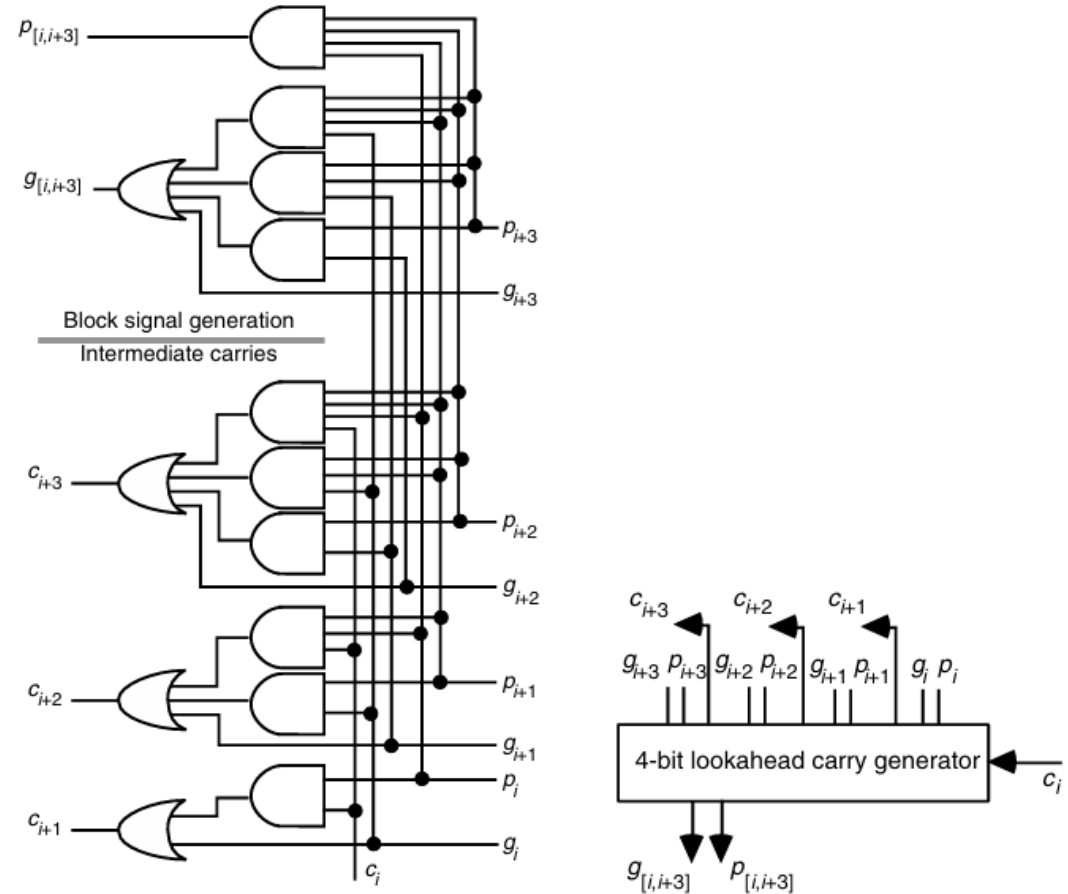
- Consider radix-16 addition of two binary numbers.
- Each number is characterized by its generate and propagate signals.
- For each radix-16-digit position:
 - Converts bit positions i to $i+3$ of the original binary numbers
 - Here, i is a multiple of 4

$$g[i, i + 3] = g_{i+3} \vee g_{i+2} \cdot p_{i+3} \vee g_{i+1} \cdot p_{i+2} \cdot p_{i+3} \vee g_i \cdot p_{i+1} \cdot p_{i+2} \cdot p_{i+3}$$

$$p[i, i + 3] = p_i \cdot p_{i+1} \cdot p_{i+2} \cdot p_{i+3}$$

Multi-Level Lookahead

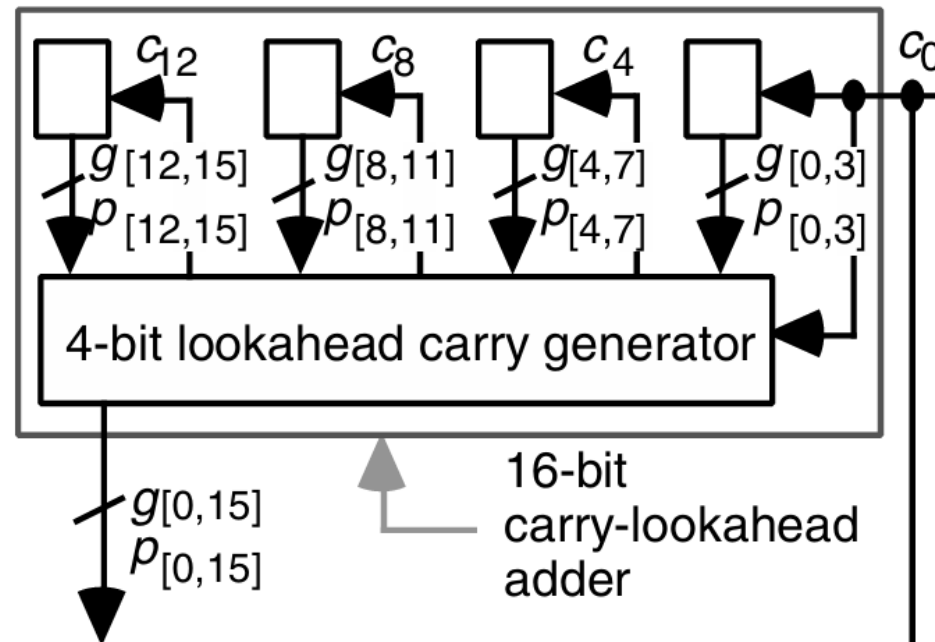
We will see shortly that such a block can be used in a multilevel structure to build a carry network of any desired width.



Multi-Level Lookahead

For example, to construct a two-level 16-bit carry-lookahead adder, we need:

- four 4-bit adders, and (may be 4-bit CLA)
- one 4-bit lookahead carry generator,



Multi-Level Lookahead

Latency of 16-bit CLA

1. Producing the g and p signals for individual bits

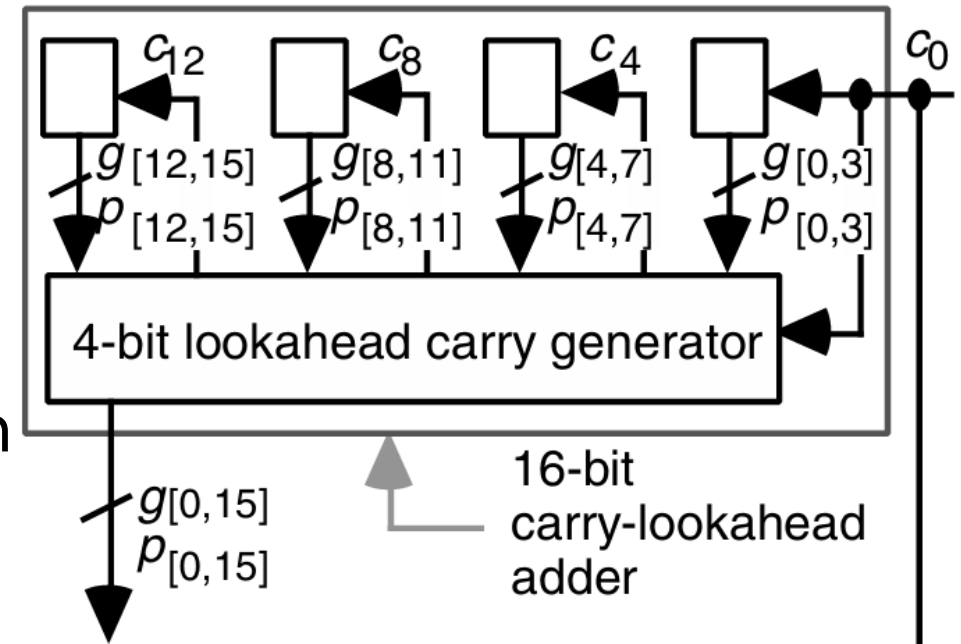
Latency: 1 gate level

2. Producing the g and p signals for each 4-bit block

Latency: 2 gate Levels

3. Predicting the block-Level Carry carries(C_4, C_8, C_{12})

Latency: 2 gate Levels



Multi-Level Lookahead

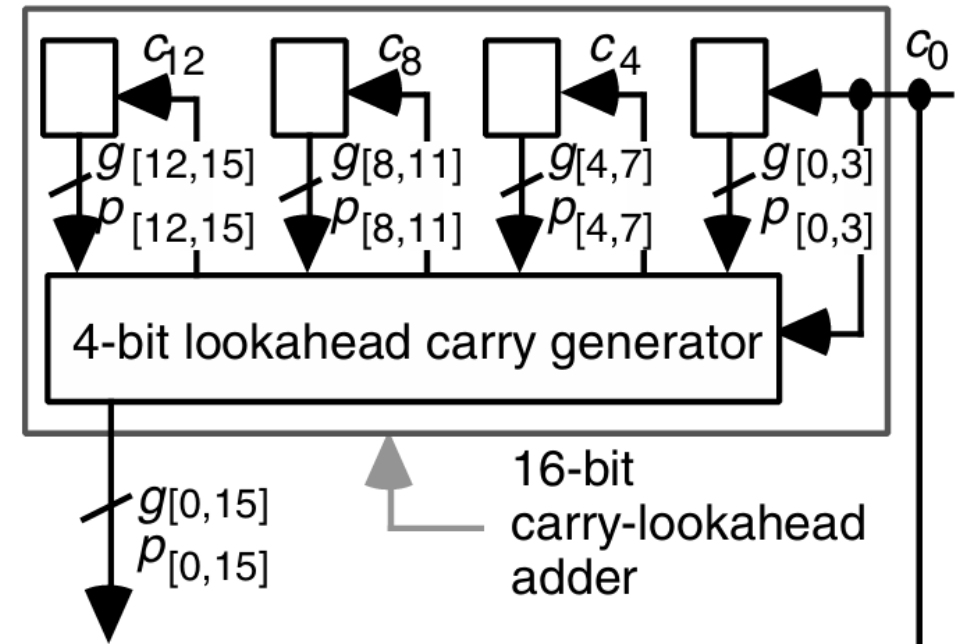
Latency of 16-bit CLA

4. Predicting The internal carries within each 4-bit block

Latency: 2 gate Levels

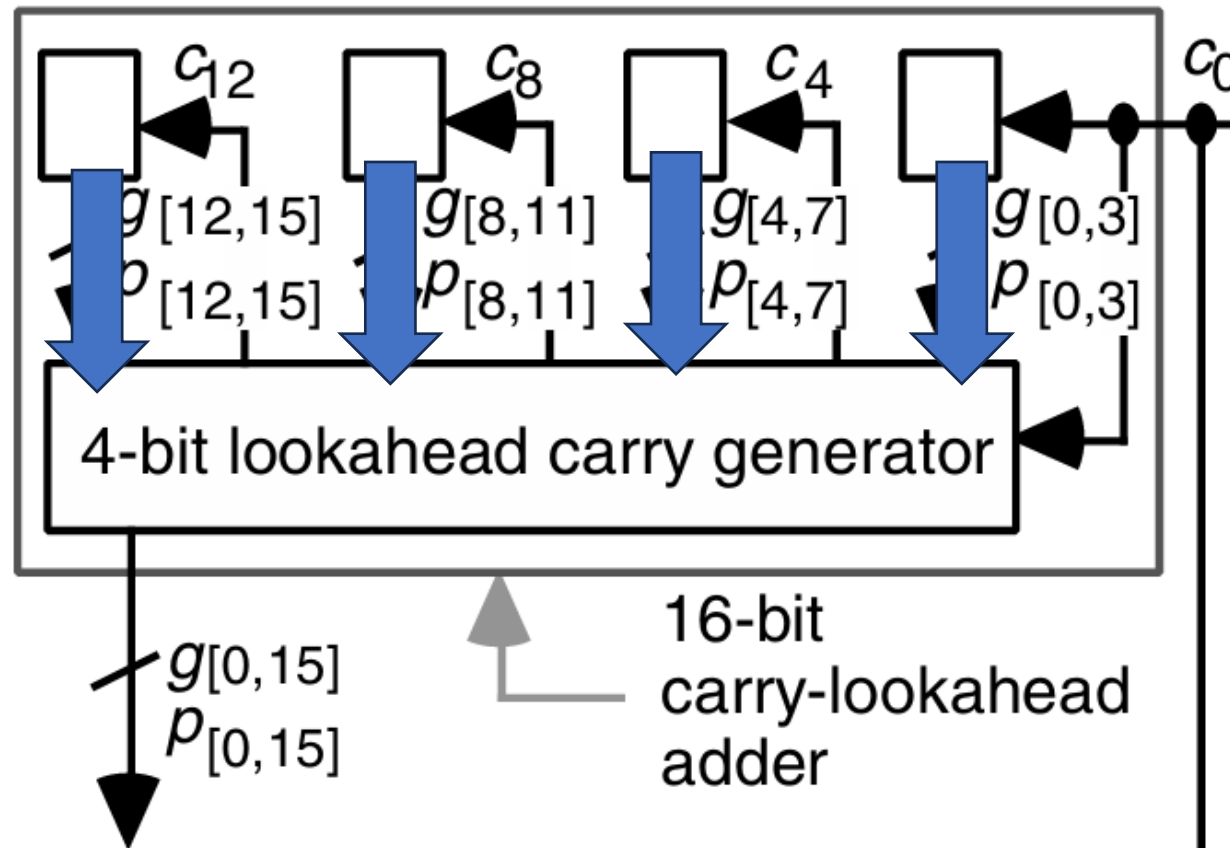
5. Computing the Sum bits

Latency: 2 gate Levels



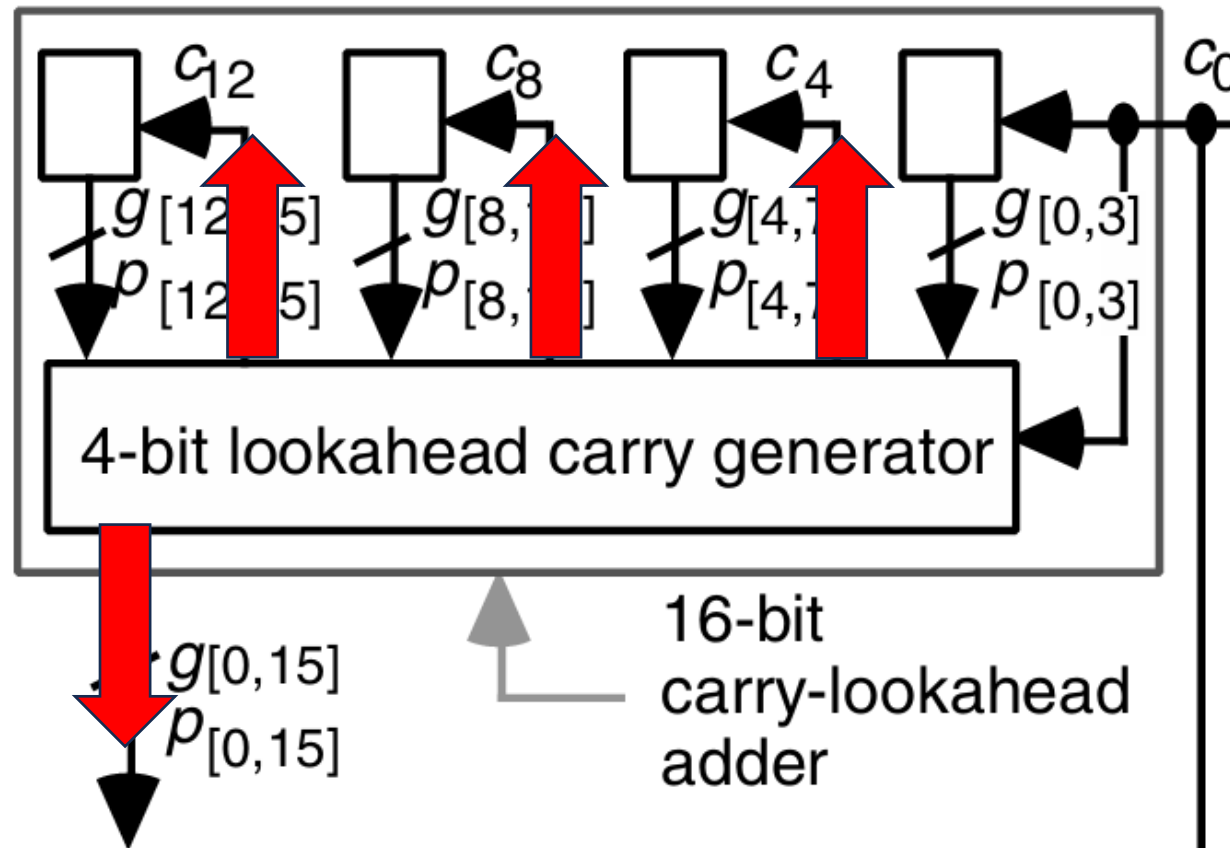
Multi-Level Lookahead

Motion of 16-bit CLA



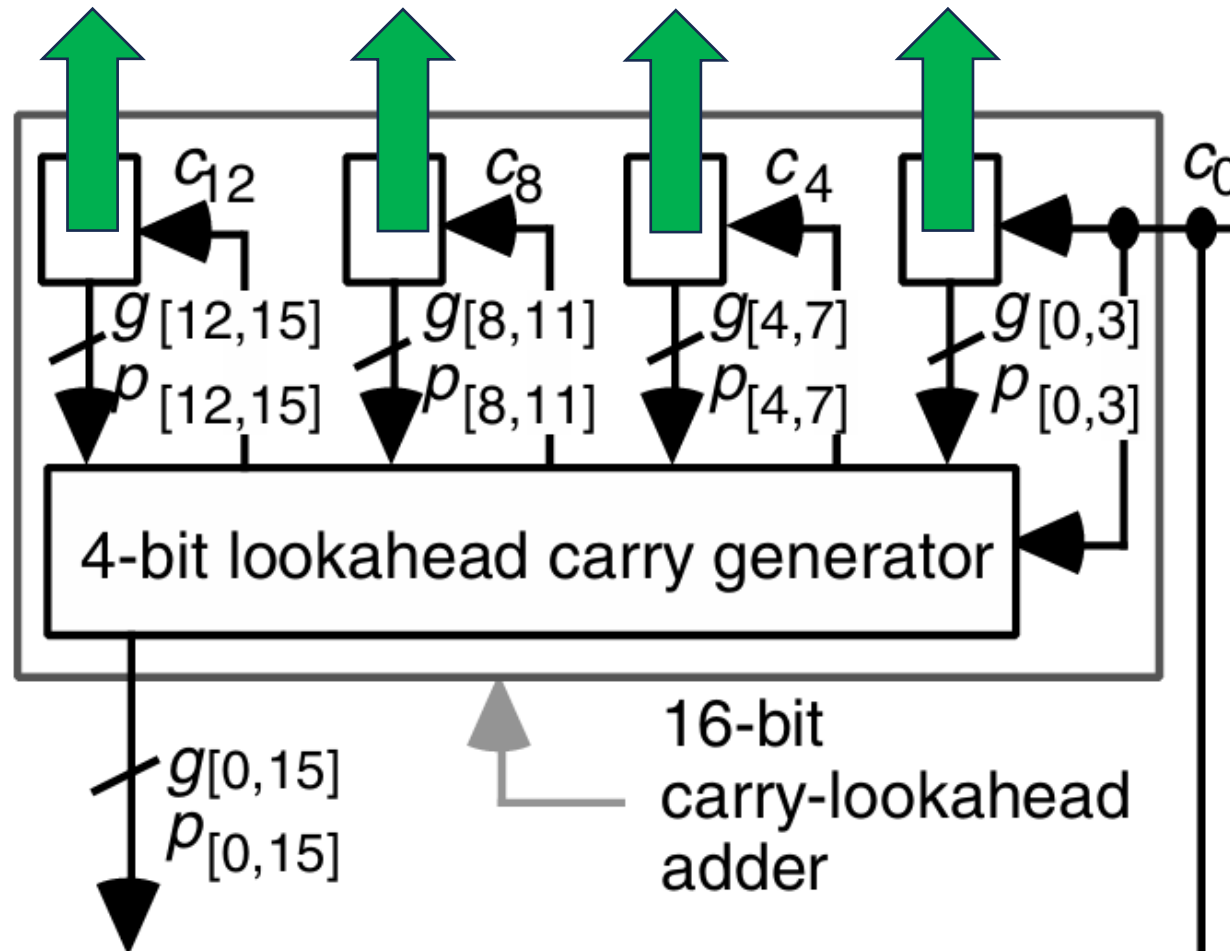
Multi-Level Lookahead

Motion of 16-bit CLA



Multi-Level Lookahead

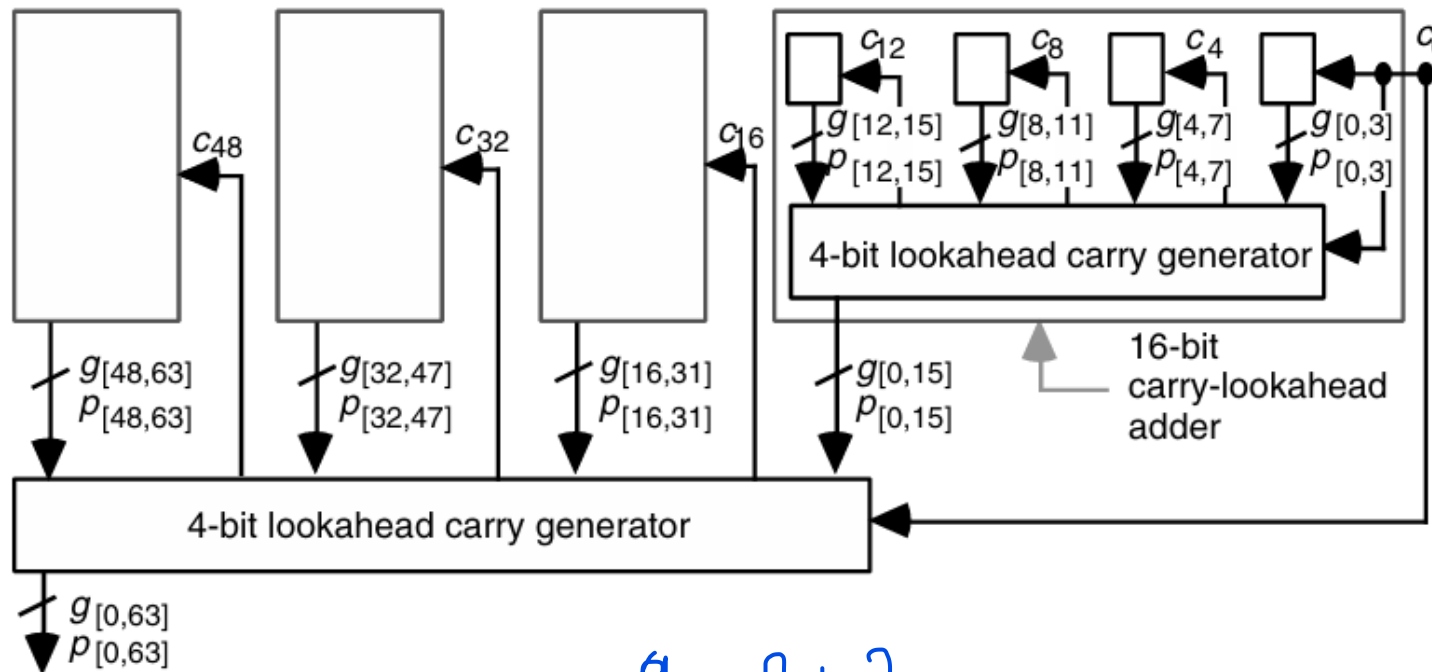
Motion of 16-bit CLA



Multi-Level Lookahead

64-Bit CLA

- By connecting 4 of 16-bit carry-lookahead adders and add one 4-bit lookahead carry generator we could make 64-bit CLA



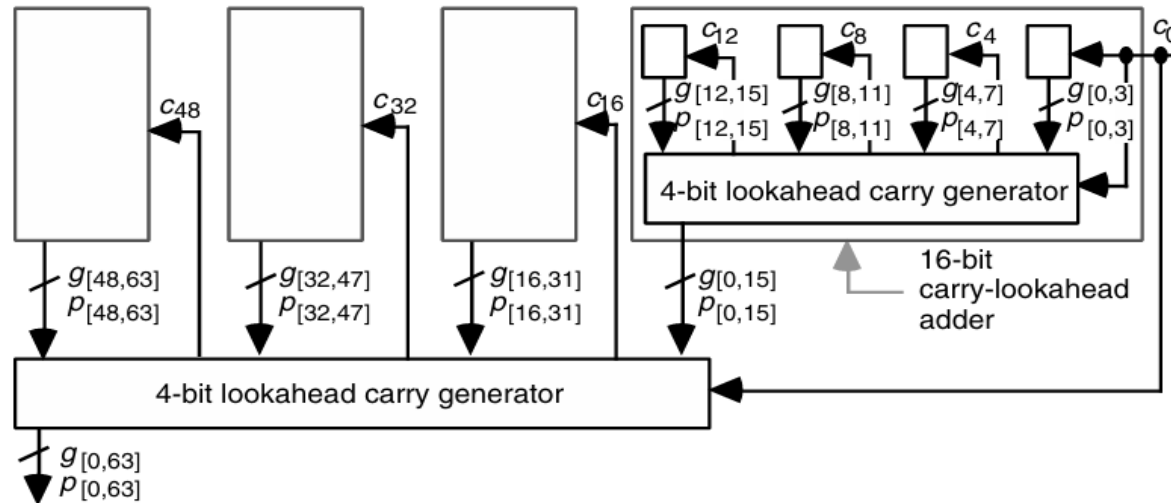
$$4 + 2 + 2$$

Multi-Level Lookahead

Latency Equation

- the total latency for the 16-bit adder is 9 gate levels, which is much better than the 32 gate levels required by a 16-bit ripple-carry adder.
- General formula for k-bit CLA (with 4-bit blocks):

$$T_{lookahead-add} = 4 \log_4 K + 1$$



Ling Adder

- Generate: $C_{out} = 1$ independent of C
 - $G = A.B$
- Propagate: $C_{out} = C$
 - $P = A \oplus B$
- Kill: $C_{out} = 0$ independent of C
 - $K = \bar{A}. \bar{B} = \overline{A \vee B}$
- $S = A \oplus B \oplus C = P \oplus C$

A	B	C	G	P	K	C_{out}	S
0	0	0	0	0	1	0	0
		1				0	1
0	1	0	0	1	0	0	1
		1				1	0
1	0	0	0	1	0	0	1
		1				1	0
1	1	0	1	0	0	1	0
		1				1	1

- $C_{out} = G \vee (P.C)$
- $C_{out} = G \vee (\bar{K}.C)$

P is XOR function and it is too slow, so I will use Kill instead of it to maximum speed between C to C_{out} .

It is also helpful to define a transfer signal corresponding to the event that the carry-out will be 1, given that the carry-in is :

$$t = G \vee P = \bar{K} = A \vee B$$

Ling Adder



Ling Adder Overview

- Ling adder = modified Carry Lookahead Adder
- Reduces hardware by propagating: (using carry instead of carry)

$$h_i = c_i \vee c_{i-1}$$

- Goal:
 - Fewer gate inputs
 - Smaller AND Gates
 - Lower hardware cost

Ling Adder

Conventional Carry

Define transfer signal:

$$c_i = g_{i-1} \vee c_{i-1}p_{i-1}$$

$$t_i = g_i \vee p_i$$

After unrolling:

$$\begin{aligned} c_i &= g_{i-1} \vee c_{i-1}p_{i-1} \\ &= g_{i-1} \vee c_{i-1}t_{i-1} \\ &= g_{i-1} \vee g_{i-2}t_{i-1} \vee g_{i-3}t_{i-2}t_{i-1} \\ &\quad \vee g_{i-4}t_{i-3}t_{i-2}t_{i-1} \vee c_{i-4}t_{i-4}t_{i-3}t_{i-2}t_{i-1} \end{aligned}$$

- Required 5-input AND gates
- Total 19 gate inputs

Ling Adder



Ling Carry Signal (Harry Signal)

- Ling proposes:

$$h_i = c_i \vee c_{i-1}$$

- let's remember:

$$c_i = g_{i-1} \vee c_{i-1}p_{i-1}$$

- Trying to replace $c_{i-1}p_{i-1}$ with harry propagation

$$\begin{aligned} c_{i-1}p_{i-1} &= c_{i-1}p_{i-1} \vee g_{i-1}p_{i-1} \text{ (Zero term)} \vee p_{i-1}c_{i-1}p_{i-1} \text{ (Repeated term)} \\ &= c_{i-1}p_{i-1} \vee p_{i-1}(g_{i-1} \vee c_{i-1}p_{i-1}) \\ &= c_{i-1}p_{i-1} \vee p_{i-1}c_i \\ &= p_{i-1}(c_{i-1} \vee c_i) \\ &= h_i p_{i-1} \end{aligned}$$

- replace c_i now

$$\begin{aligned} c_i &= g_{i-1} \vee c_{i-1}p_{i-1} \\ &= g_{i-1}h_i \text{ [doesn't affect in the result]} \vee h_i p_{i-1} \\ &= h_i(g_{i-1} \vee p_{i-1}) \\ &= h_i t_{i-1} \end{aligned}$$

Ling Adder

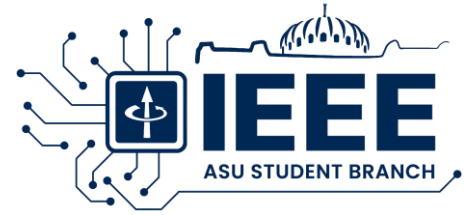
Ling Recurrence

$$\begin{aligned}h_i &= c_i \vee c_{i-1} \\&= g_{i-1} \vee c_{i-1}p_{i-1} \vee c_{i-1} \\&= g_{i-1} \vee c_{i-1} \\&= g_{i-1} \vee h_{i-1}t_{i-2}\end{aligned}$$

- Unrolling the preceding recurrence for h_i

$$\begin{aligned}h_i &= g_{i-1} \vee t_{i-2}h_{i-1} \\&= g_{i-1} \vee t_{i-2}(g_{i-2} \vee h_{i-2}t_{i-3}) \\&= g_{i-1} \vee g_{i-2} \vee h_{i-2}t_{i-2}t_{i-3} \quad (\text{since } t_{i-2}g_{i-2} = g_{i-2}) \\&= g_{i-1} \vee g_{i-2} \vee g_{i-3}t_{i-3}t_{i-2} \vee h_{i-3}t_{i-4}t_{i-3}t_{i-2} \\&= g_{i-1} \vee g_{i-2} \vee g_{i-3}t_{i-2} \vee g_{i-4}t_{i-3}t_{i-2} \vee h_{i-4}t_{i-4}t_{i-3}t_{i-2}\end{aligned}$$

Ling Adder



Hardware Comparison

- Using h_i
 - 5 Product terms
 - Max AND Size = 4
 - Total inputs = 14

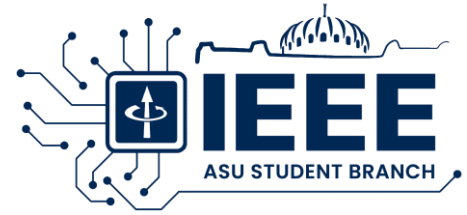
$$\begin{aligned}h_i &= g_{i-1} \vee h_{i-1}t_{i-2} \\ &= g_{i-1} \vee g_{i-2} \vee g_{i-3}t_{i-2} \vee g_{i-4}t_{i-3}t_{i-2} \vee h_{i-4}t_{i-4}t_{i-3}t_{i-2}\end{aligned}$$

- Using C_i
 - 5 Product terms
 - Max AND Size = 5
 - Total inputs = 19

$$\begin{aligned}c_i &= g_{i-1} \vee c_{i-1}t_{i-1} \\ &= g_{i-1} \vee g_{i-2}t_{i-1} \vee g_{i-3}t_{i-2}t_{i-1} \vee g_{i-4}t_{i-3}t_{i-2}t_{i-1} \vee c_{i-4}t_{i-4}t_{i-3}t_{i-2}t_{i-1}\end{aligned}$$

- Ling Adder uses smaller gates
- Better inputs
- Better for wired-OR

Ling Adder



Complex Sum expression

- Carry recovered by:

$$c_i = h_i t_{i-1}$$

- Sum:

$$s_i = p_i \oplus c_i = p_i \oplus h_i t_{i-1}$$

- The advantage of transfer signal based that it is OR based and there is no XOR gates, unless in sum calculation but it is not that important in critical path

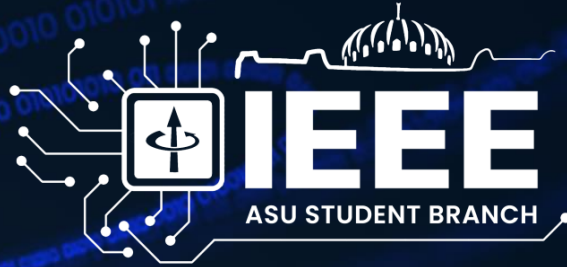
Example h signals

$$h_1 = g_0 \vee c_{in}$$

$$h_2 = g_1 \vee t_0 h_1$$

$$h_3 = g_2 \vee g_1 \vee t_1 t_0 h_1$$

$$h_4 = g_3 \vee g_2 \vee g_1 t_2 \vee t_0 t_1 t_2 h_1$$



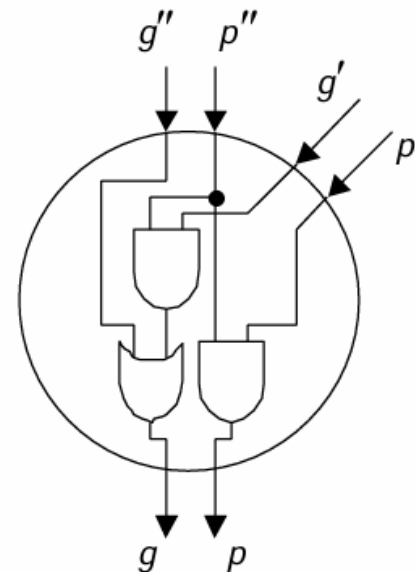
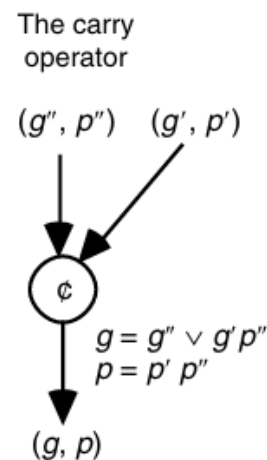
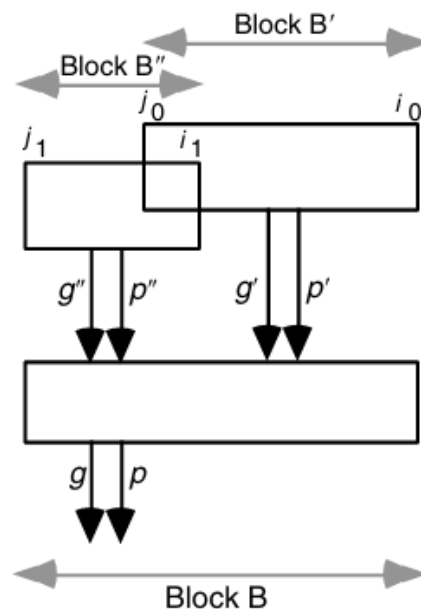
Adders

Carry-Lookahead Adders: Prefix-Based Carry Network

Prefix Computation

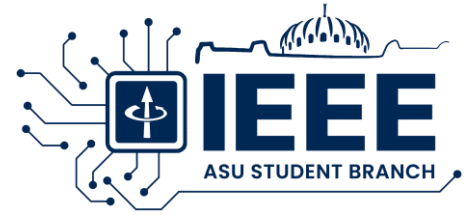
Carry Determination as Prefix Computation

- That is, carry generation in the larger group takes place if the left group generates a carry or the right group generates a carry and the left one propagates it, while propagation occurs if both groups propagate the carry.



$$g'' \vee g' p''$$

Prefix Computation



Carry Determination as Prefix Computation

$(g, p) = (g', p') \text{ } \text{ } (g'', p'')$ means $g = g'' \vee g'p''$, $p = p'p''$

Given

(g_0, p_0) (g_1, p_1) $\cdots$ (g_{k-2}, p_{k-2}) (g_{k-1}, p_{k-1})

Find

$(g_{[0,0]}, p_{[0,0]})$ $(g_{[0,1]}, p_{[0,1]})$ $\cdots$ $(g_{[0,k-2]}, p_{[0,k-2]})$ $(g_{[0,k-1]}, p_{[0,k-1]})$

The desired signal pairs can be obtained by evaluating all the prefixes of

$$(g_0, p_0) \text{ } \text{ } (g_1, p_1) \text{ } \text{ } \cdots \text{ } \text{ } (g_{k-2}, p_{k-2}) \text{ } \text{ } (g_{k-1}, p_{k-1})$$

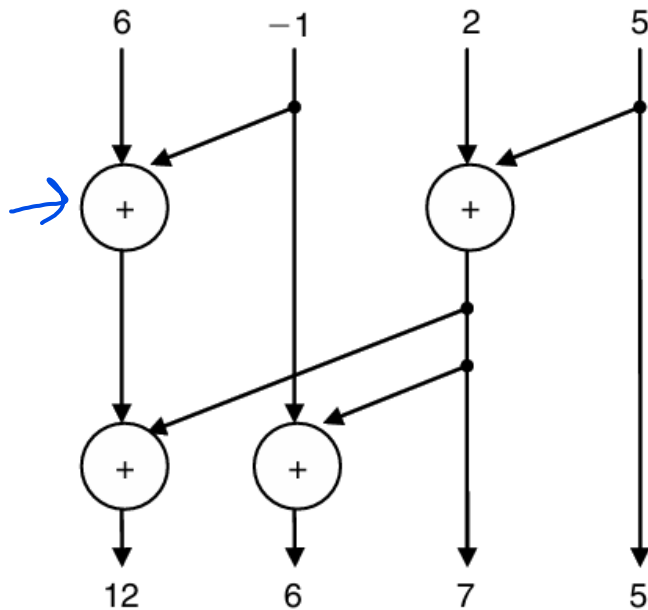
Given:	x_0	x_1	x_2	x_3	$\cdots$	x_{k-1}
Find:	x_0	$x_0 + x_1$	$x_0 + x_1 + x_2$	$x_0 + x_1 + x_2 + x_3$	$\cdots$	$x_0 + x_1 + \cdots + x_{k-1}$

Parallel Prefix Networks

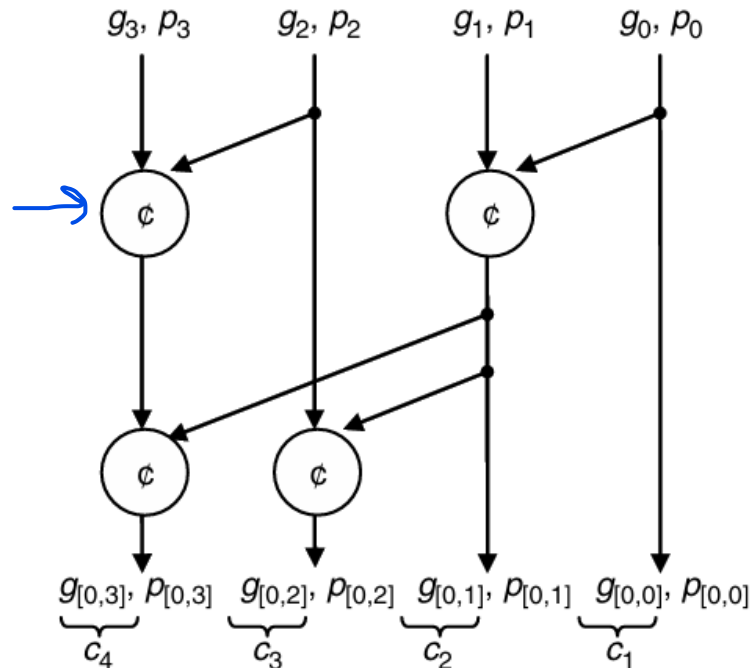
Prefix Computation

Carry Determination as Prefix Computation

$(g, p) = (g', p') \phi (g'', p'')$ means $g = g' \vee g'p''$, $p = p'p''$



(a) Prefix sums network



(b) Carry network

(no waiting)

Parallel Prefix Networks

Prefix Computation

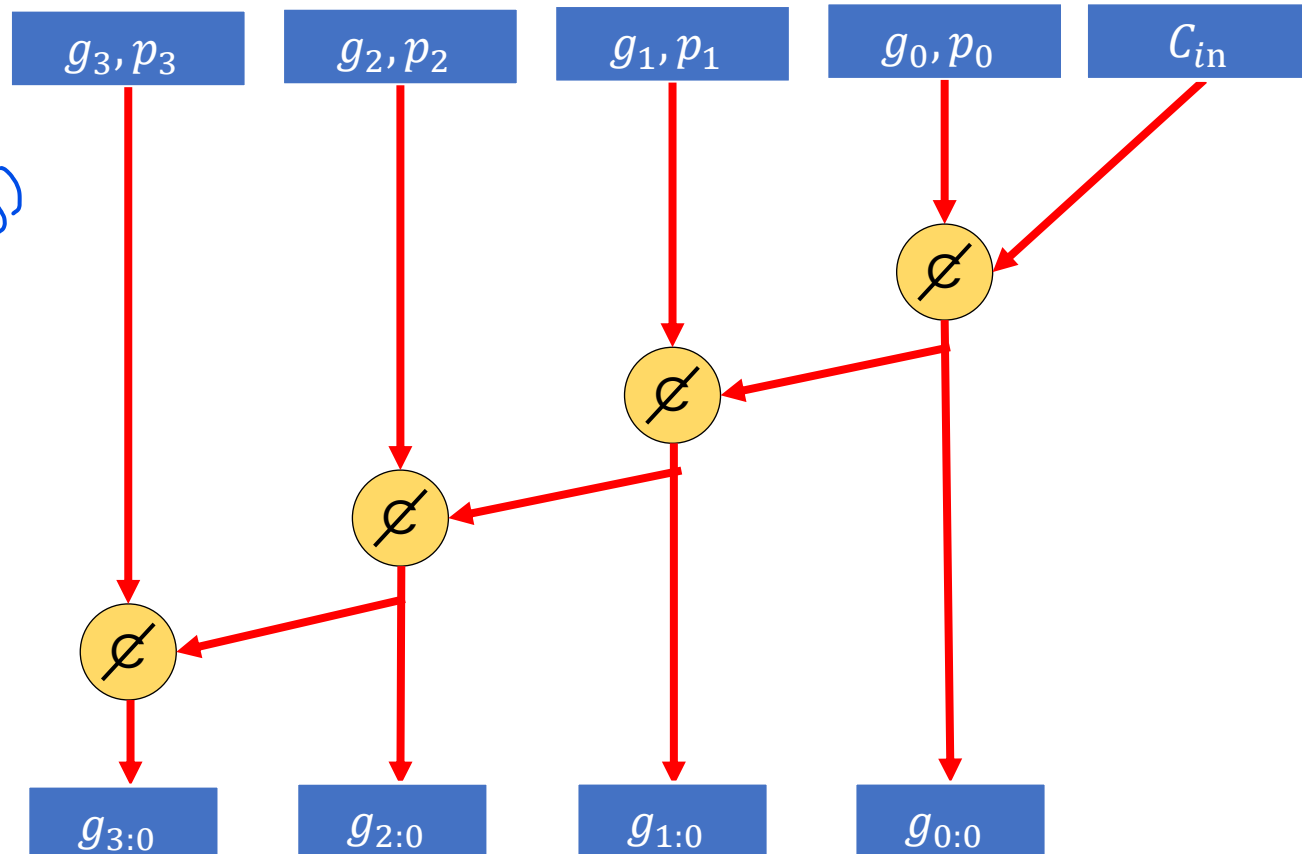
How to get this Carry Network: (Example 1)

$$\begin{aligned}g_{[3:0]} &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_0 \vee p_3 p_2 p_1 p_0 g_{-1} \\&= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee \underline{p_3 p_2 p_1 g_0 \vee p_3 p_2 p_1 p_0 g_{-1}} \\&= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 (g_0 \vee p_0 g_{-1}) \\(g_0 \vee p_0 g_{-1}) &= g_{[0:0]} = c_0 \\&= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_{[0:0]} \\&= g_3 \vee p_3 g_2 \vee \underline{p_3 p_2 g_1 \vee p_3 p_2 p_1 g_{[0:0]}} \\&= g_3 \vee p_3 g_2 \vee p_3 p_2 (g_1 \vee p_1 g_{[0:0]}) \\(g_1 \vee p_1 g_{[0:0]}) &= g_{[1:0]} = c_1 \\&= g_3 \vee p_3 g_2 \vee p_3 p_2 g_{[1:0]} \\&= g_3 \vee \underline{p_3 g_2 \vee p_3 p_2 g_{[1:0]}} \\&= g_3 \vee p_3 (g_2 \vee p_2 g_{[1:0]}) \\(g_2 \vee p_2 g_{[1:0]}) &= g_{[2:0]} = c_2 \\&= g_3 \vee p_3 g_{[2:0]} \\(g_3 \vee p_3 g_{[2:0]}) &= g_{[3:0]} = c_3 \\&= g_{[3:0]}\end{aligned}$$

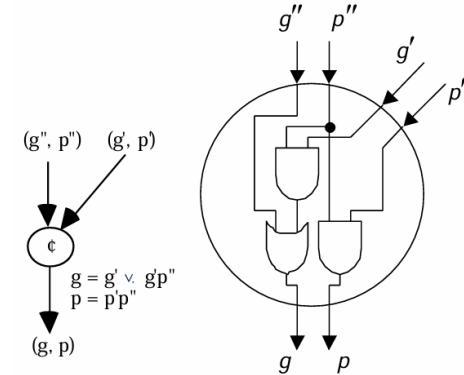
Prefix Computation

How to get this Carry Network: (Example 1)

The result:



✓
(Waiting)



$$\begin{aligned}
 g_{[3:0]} &= g_3 \vee p_3g_2 \vee p_3p_2g_1 \vee p_3p_2p_1g_0 \vee p_3p_2p_1p_0g_{-1} \\
 &= g_3 \vee p_3g_2 \vee p_3p_2g_1 \vee \underline{p_3p_2p_1g_0} \vee \underline{p_3p_2p_1p_0g_{-1}} \\
 &= g_3 \vee p_3g_2 \vee p_3p_2g_1 \vee p_3p_2p_1(g_0 \vee p_0g_{-1}) \\
 (g_0 \vee p_0g_{-1}) &= g_{[0:0]} = c_0 \\
 &= g_3 \vee p_3g_2 \vee p_3p_2g_1 \vee p_3p_2p_1g_{[0:0]} \\
 &= g_3 \vee p_3g_2 \vee \underline{p_3p_2g_1} \vee \underline{p_3p_2p_1g_{[0:0]}} \\
 &= g_3 \vee p_3g_2 \vee p_3p_2(g_1 \vee p_1g_{[0:0]}) \\
 (g_1 \vee p_1g_{[0:0]}) &= g_{[1:0]} = c_1 \\
 &= g_3 \vee p_3g_2 \vee p_3p_2g_{[1:0]} \\
 &= g_3 \vee p_3g_2 \vee \underline{p_3p_2g_{[1:0]}} \\
 &= g_3 \vee p_3(g_2 \vee p_2g_{[1:0]}) \\
 (g_2 \vee p_2g_{[1:0]}) &= g_{[2:0]} = c_2 \\
 &= g_3 \vee p_3g_{[2:0]} \\
 (g_3 \vee p_3g_{[2:0]}) &= g_{[3:0]} = c_3 \\
 &= g_{[3:0]}
 \end{aligned}$$

Parallel Prefix Networks

Prefix Computation

How to get this Carry Network: (Example 2)

$$\begin{aligned}g_{[3:0]} &= g_3 \vee p_3g_2 \vee p_3p_2g_1 \vee p_3p_2p_1g_0 \vee p_3p_2p_1p_0g_{-1} \\&= \underline{g_3 \vee p_3g_2} \vee p_3p_2g_1 \vee \underline{p_3p_2p_1g_0 \vee p_3p_2p_1p_0g_{-1}} \\&= \underline{g_3 \vee p_3g_2} \vee p_3p_2g_1 \vee p_3p_2p_1(g_0 \vee p_0g_{-1})\end{aligned}$$

$$g_{[3:2]} = (g_3 \vee p_3g_2), p_{3:2} = p_3p_2$$

$$(g_0 \vee p_0g_{-1}) = g_{[0:0]} = c_0$$

$$g_{[3:0]} = g_{[3:2]} \vee \underline{p_3p_2g_1 \vee p_3p_2p_1g_{[0:0]}}$$

$$(g_1 \vee p_1g_{[0:0]}) = g_{[1:0]} = c_1$$

$$g_{[3:0]} = g_{[3:2]} \vee p_3p_2g_{[1:0]}$$

another analogy for $g_{[3:0]}$

$$g_{[3:0]} = g_{[3:2]} \vee p_{[3:2]}g_{[1:0]}$$

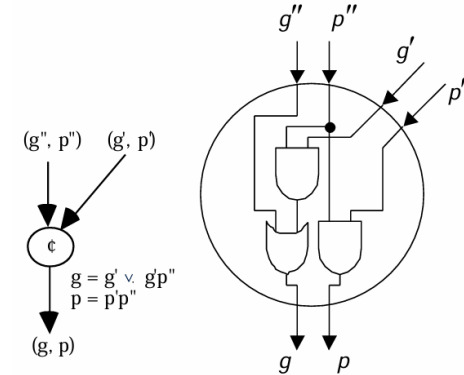
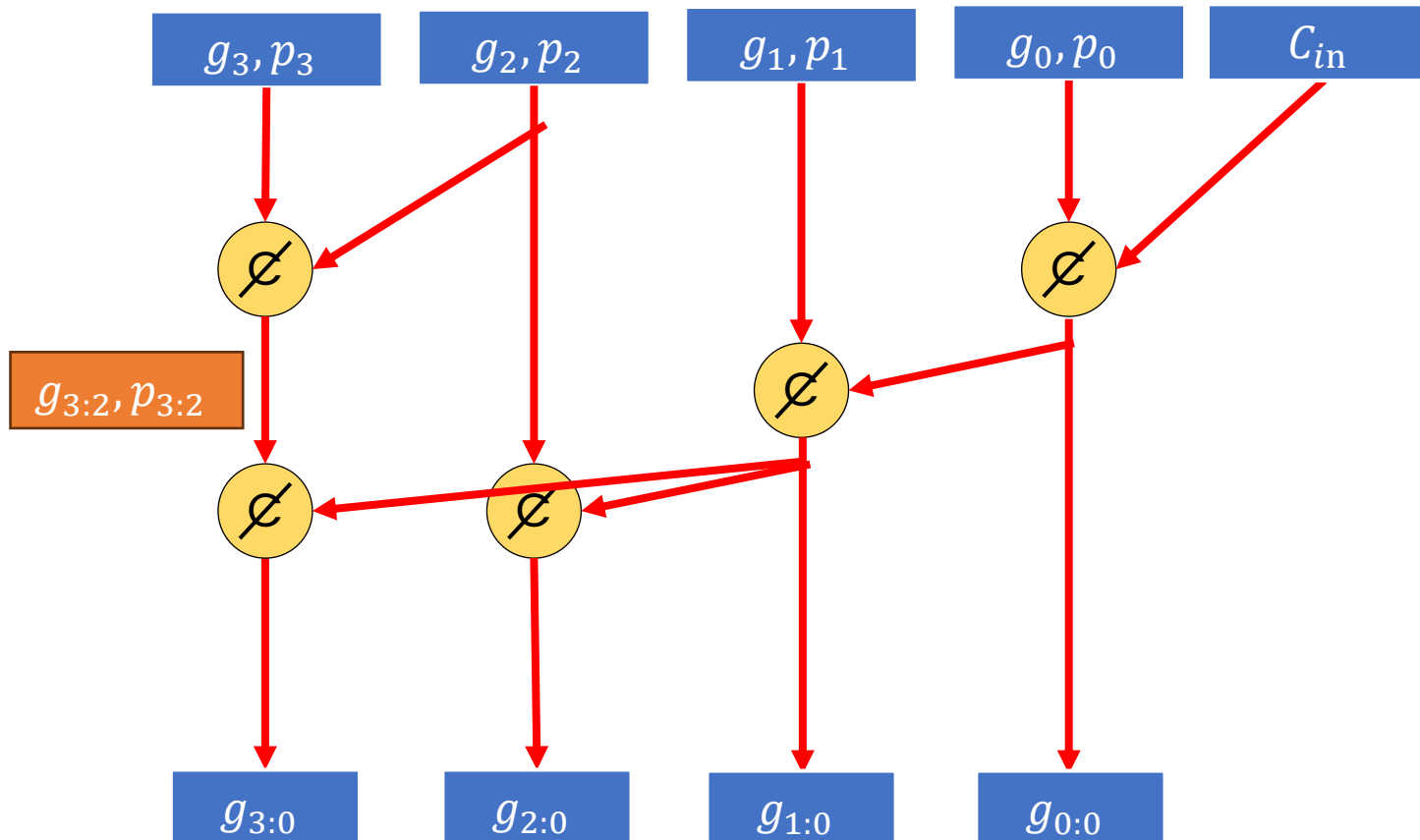
$$\text{and for } g_{[2:0]} = g_2 \vee p_2g_{[1:0]}$$

no waiting!

Prefix Computation

How to get this Carry Network: (Example 2)

The result:



$$\begin{aligned}
 g_{[3:0]} &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_0 \vee p_3 p_2 p_1 p_0 g_{-1} \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_0 \vee p_3 p_2 p_1 p_0 g_{-1} \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 (g_0 \vee p_0 g_{-1}) \\
 (g_0 \vee p_0 g_{-1}) &= g_{[0:0]} = c_0 \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_{[0:0]} \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_1 \vee p_3 p_2 p_1 g_{[0:0]} \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 (g_1 \vee p_1 g_{[0:0]}) \\
 (g_1 \vee p_1 g_{[0:0]}) &= g_{[1:0]} = c_1 \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_{[1:0]} \\
 &= g_3 \vee p_3 g_2 \vee p_3 p_2 g_{[1:0]} \\
 &= g_3 \vee p_3 (g_2 \vee p_2 g_{[1:0]}) \\
 (g_2 \vee p_2 g_{[1:0]}) &= g_{[2:0]} = c_2 \\
 &= g_3 \vee p_3 g_{[2:0]} \\
 (g_3 \vee p_3 g_{[2:0]}) &= g_{[3:0]} = c_3 \\
 &= g_{[3:0]}
 \end{aligned}$$

Brent and Kung



The Brent-Kung Recursive Construction

- The inputs are first combined pairwise to obtain the following sequence of length $k/2$:

$$x_0 + x_1$$

$$x_2 + x_3$$

$$x_4 + x_5$$

...

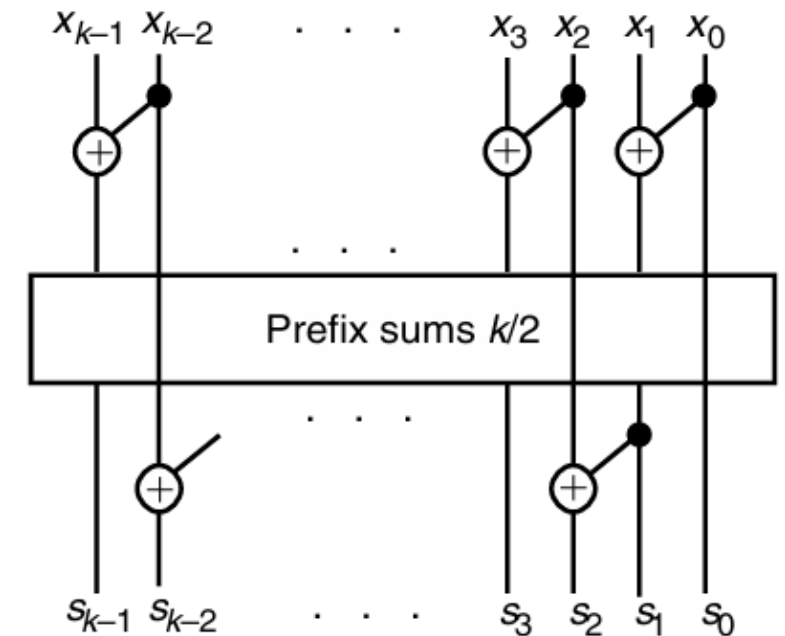
$$x_{k-4} + x_{k-3}$$

$$x_{k-2} + x_{k-1}$$

Brent and Kung

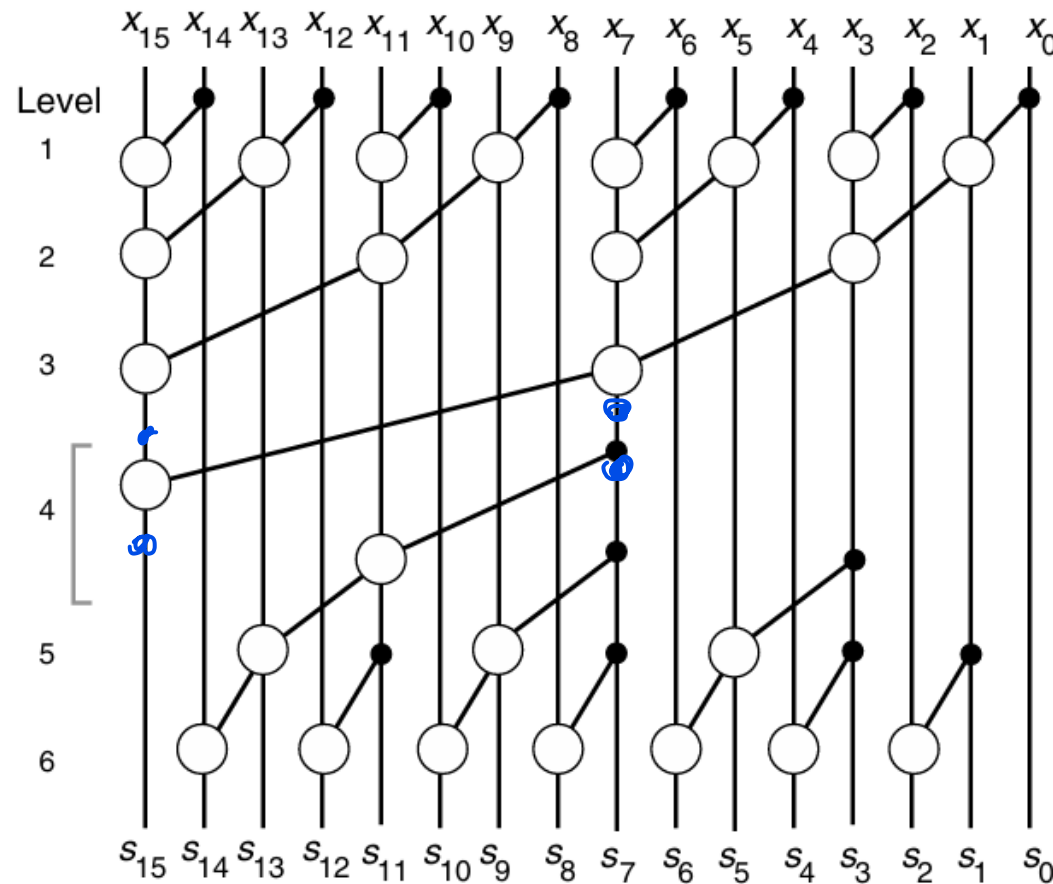
The Brent-Kung Recursive Construction

- Computes prefixes for 2-bit groups, then used to find prefixes for 4-bit groups, then for 8-bit groups, and so forth.
 - To this to till only have one group
- Downward sweep
 - Same as Upward but from highest to lowest
- Final Stage: $s_{2j} = s_{2j-1} + x_{2j}$



Brent and Kung

Cost and Delay Equations



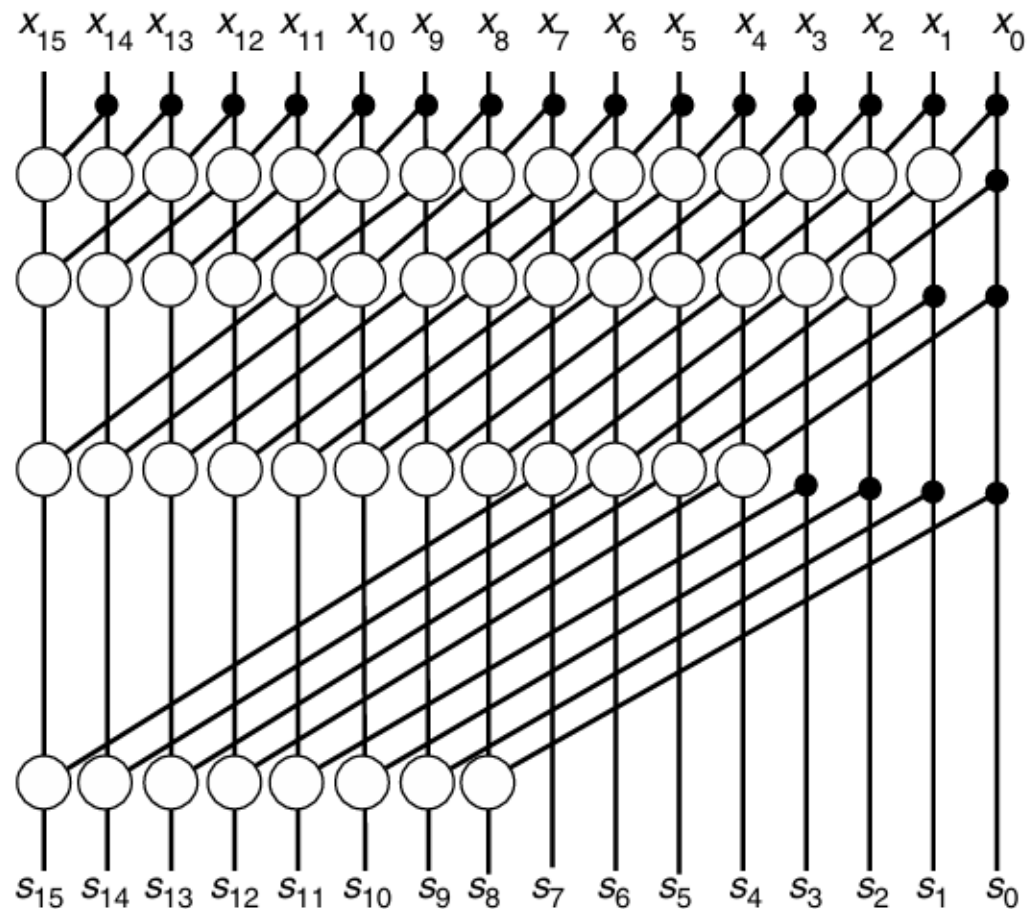
Delay: $D(k) = D(k/2) + 2 = 2\log_2 k - 2$

Cost: $C(k) = C(k/2) + k - 1 = 2k - 2 - \log_2 k$

Fanout is 2 at each stage
(only one wiring track between levels)

Kogge-Stone

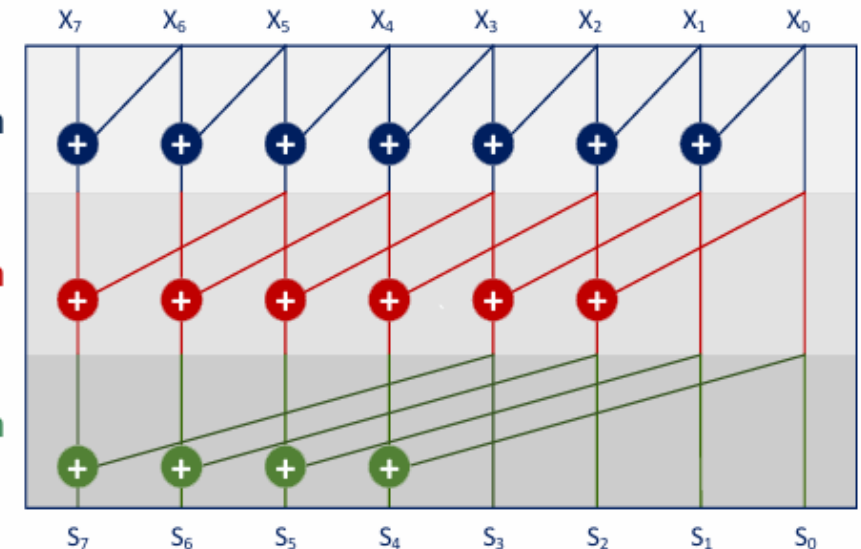
Kogge-Stone Algorithm



Radix-2 prefix sum
are all computed

Radix-4 prefix sum
are all computed

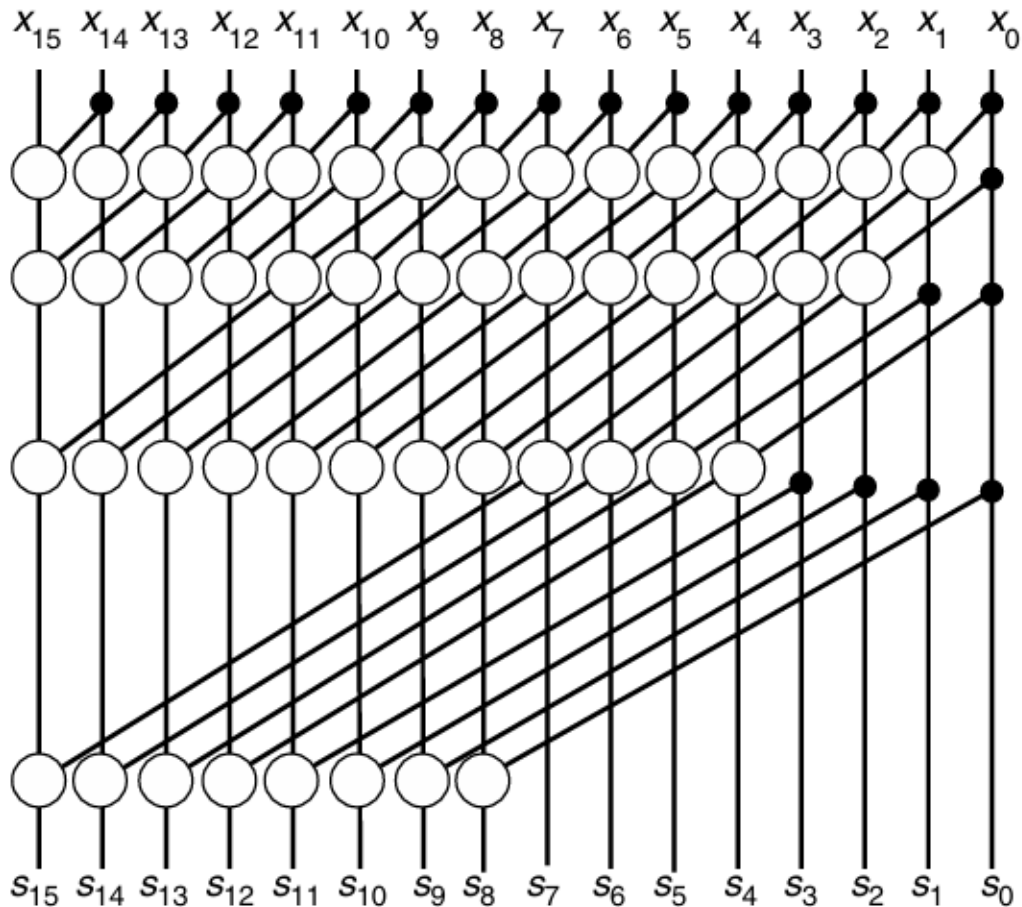
Radix-8 prefix sum
are all computed



Parallel Prefix Networks

Kogge-Stone

Cost and Delay Equations



Delay: $D(k) = D(K/2) = \log_2 k$

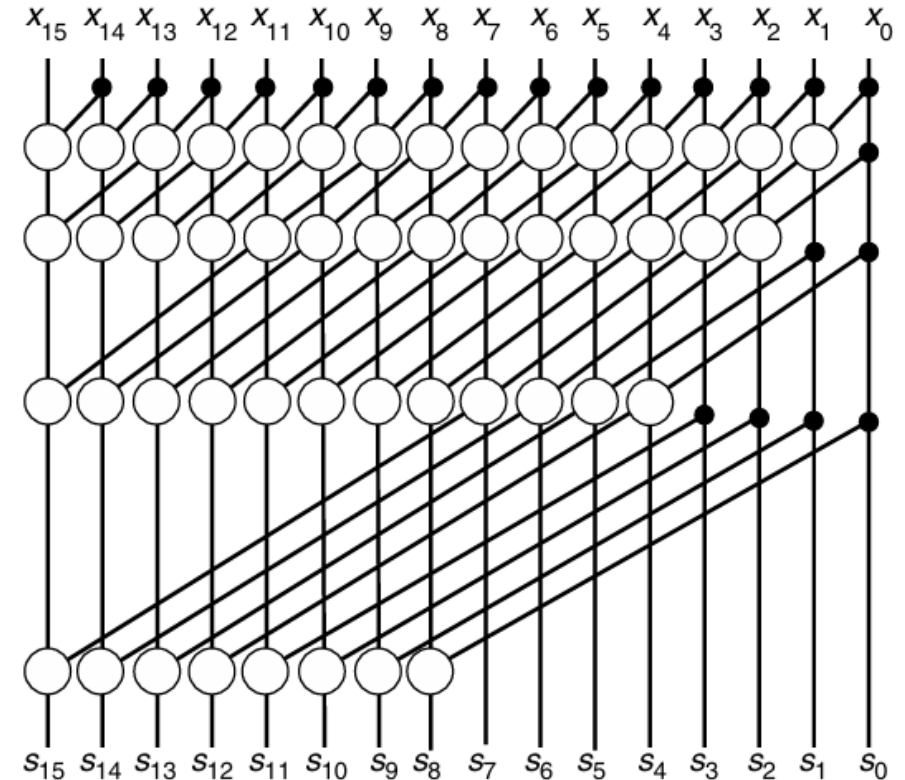
Cost: $C(k) = k \cdot \log_2 k - k + 1$

Fanout is 2 at each stage
Too many long wires

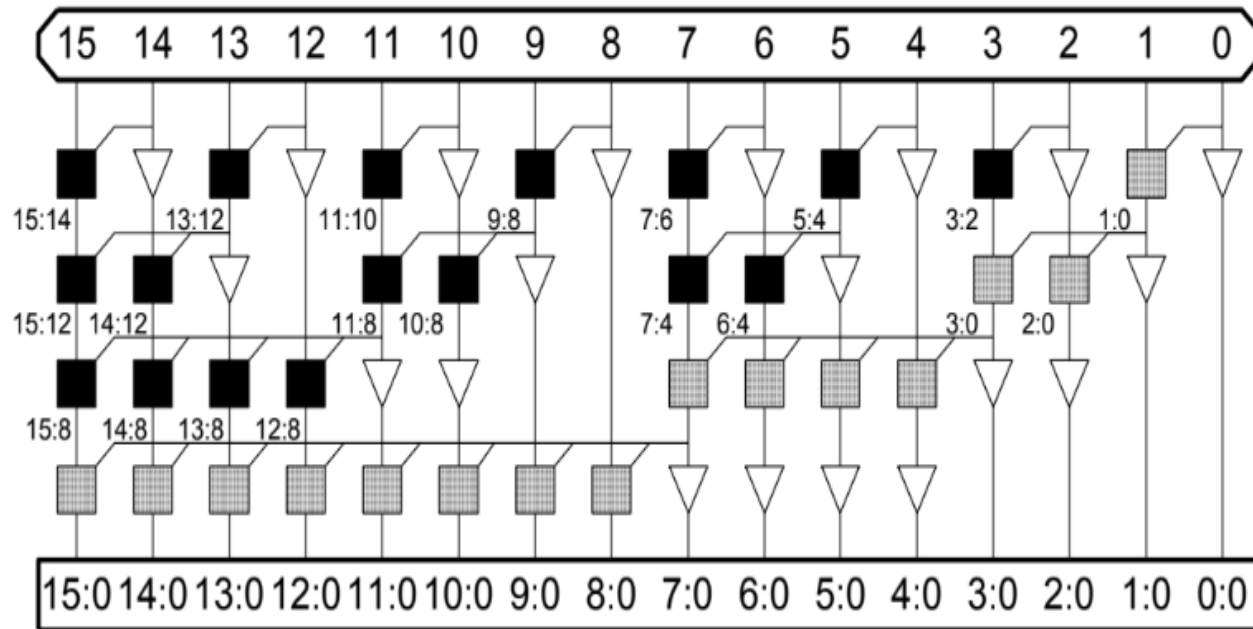
Parallel Prefix Networks

Kogge-Stone

The Kogge-Stone parallel prefix graph represents the fastest possible implementation of a parallel prefix computation if only two-input blocks are allowed. However, its cost can be prohibitive for large k , in terms of both the number of cells and the dense wiring between them. Widely used in high-performance 32-bit and 64-bit adders, but too many wires.



Cost and Delay Equations



Delay: $D(k) = D(K/2) = \log_2 k$

Lower Cost than Kogge-Stone

But fanouts double at each level [1, 2, 4, 8]

Tree Adder Taxonomy



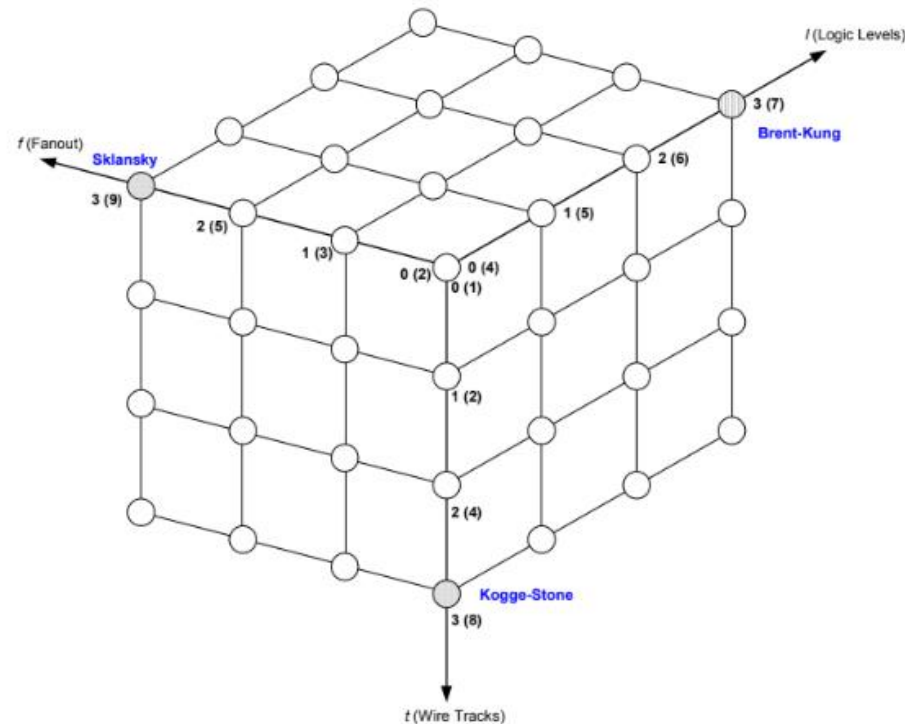
Describe adder with 3D Taxonomy (l,f,t)

- Ideal N-Bit tree would have
 - $L = \log_2 N$ levels
 - Fanout exceeding 2
 - No more than one wiring track between levels
- Logic levels: $L+1$
 - L is no. of levels more than $\log_2 N$
- Fanout: 2^{f+1}
 - f is zero for fanout of 2
- Wiring tracks: 2^t
 - t is zero for one wiring track between levels

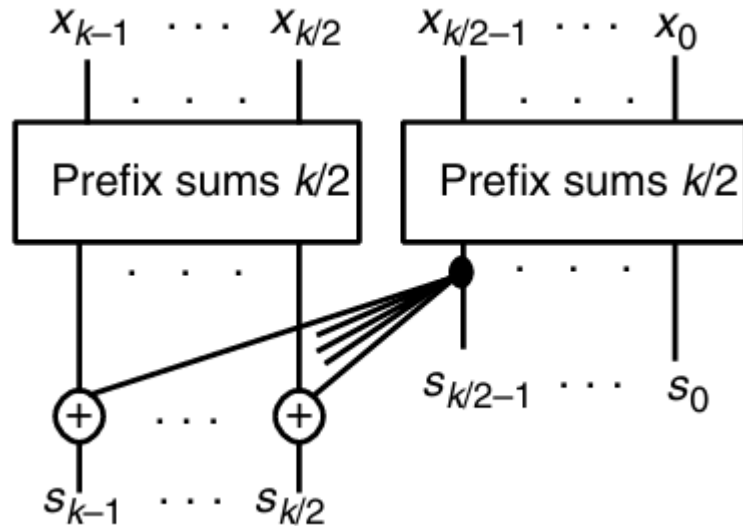
Tree Adder Taxonomy

Describe adder with 3D Taxonomy (l,f,t)

- Known tree adders sit on plane defined by: $l + f + t = L - 1$
- Brent-Kung has too many logic levels, Sklansky has too much fanout, and Kogge-Stone has too many wires



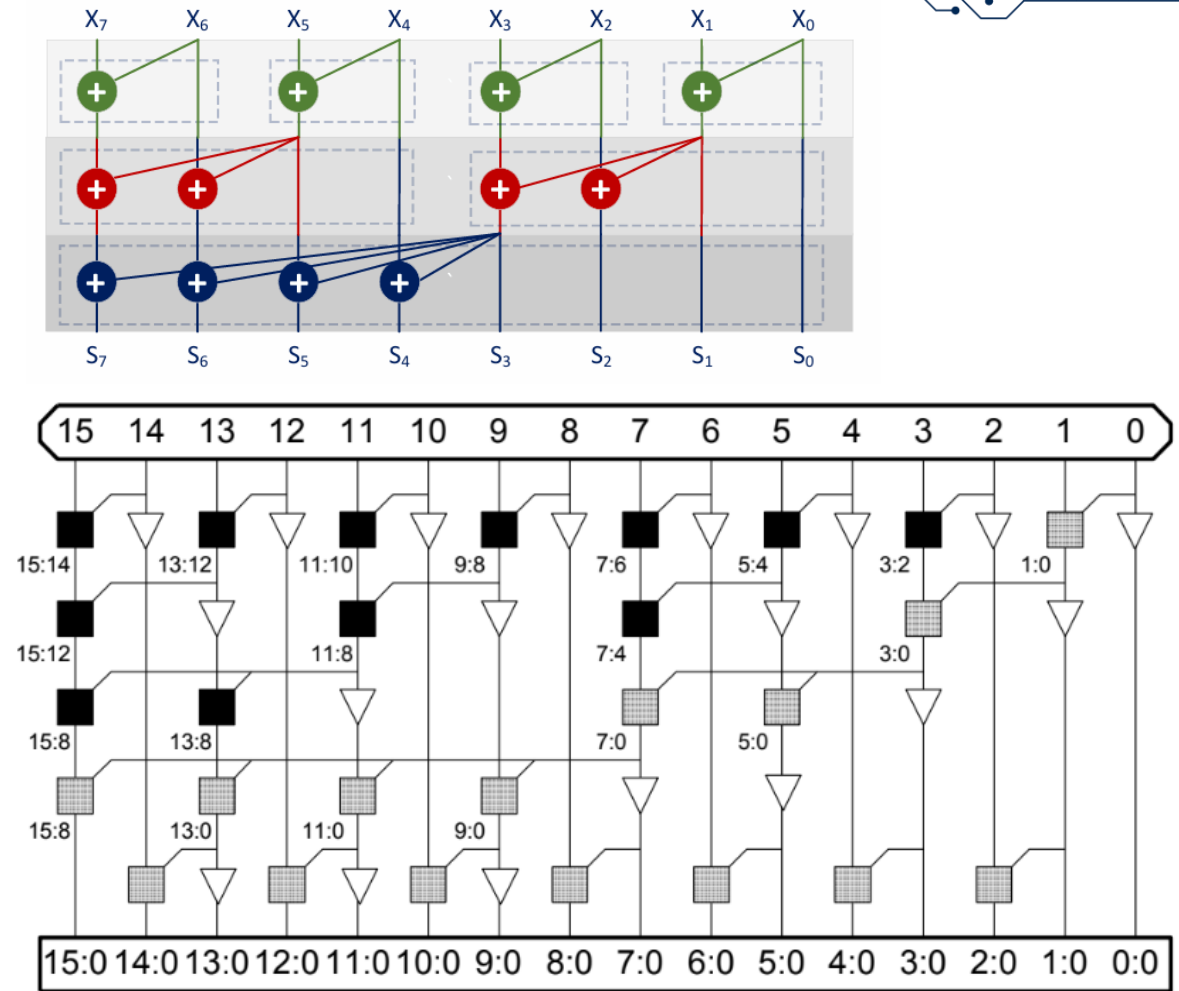
Ladner-Fischer



Delay: $D(k) = D(K/2) + 1 = \log_2 k + 1$

Cost: $C(k) = 2C(K/2) + k/2 = (k/2) \log_2 k$

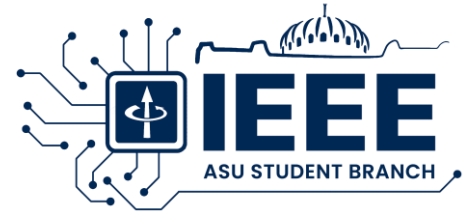
Fanout higher than 2
Too many



Parallel Prefix Networks

Thank You

References



- B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed., Oxford University Press, New York/Oxford, 2010.
- B. Parhami, *ECE 252B Course Slides*, Chapters 5–7, University of California, Santa Barbara.
- H. A. Omran, *Digital IC Design – Lecture 26: Adders*, Integrated Circuits Laboratory (ICL), Electronics and Communications Engineering Dept., Faculty of Engineering, Ain Shams University. (Based mainly on N. Weste and D. Harris, *CMOS VLSI Design*, 4th ed.)
- Abdelsalam, *Computer Arithmetic – Adders Session*, STMicroelectronics Training Slides.
- *Demo of the Pentium FDIV Bug*, YouTube video: https://www.youtube.com/watch?v=2OgP2WH5I_0
- I. Koren, *Computer Arithmetic Algorithms*, 2nd ed., A. K. Peters, Natick, MA, 2002, ISBN: 978-1568811604.